

PROMETEO



Lectarium

Desarrollo de Aplicaciones Web

Andrea de Gregorio Rodríguez

Cristina Fernández Tornel

Tutor: Raúl Albiol

ÍNDICE

Contenido

RESUMEN	3
ABSTRACT	3
JUSTIFICACIÓN DEL PROYECTO	4
INTRODUCCIÓN	7
OBJETIVOS	9
DESCRIPCIÓN	21
DISEÑOS.....	27
TECNOLOGÍAS	42
METODOLOGÍA	50
CONCLUSIONES.....	54
TRABAJOS FUTUROS	56
REFERENCIAS	58

RESUMEN

Lectarium es una plataforma web centrada en una biblioteca virtual desarrollada como Proyecto Intermodular del ciclo formativo de grado superior de Desarrollo de Aplicaciones Web. La aplicación permite a los usuarios registrarse en la plataforma y acceder a un catálogo de libros digitales, solicitar préstamos para leer los libros durante un plazo de veinte días y marcar sus favoritos, y a su vez ver su historial de préstamos y todos sus favoritos desde un panel de usuario. Todo ello sin restricciones de horario ni de ubicación.

La arquitectura del proyecto sigue el patrón cliente-servidor visto en el ciclo formativo. El frontend está desarrollado con HTML5, CSS3 y JavaScript, el backend con Node.js y Express.js, y la base de datos con MySQL. La autenticación de usuarios se implementa mediante contraseñas hashadas con bcrypt y tokens JWT, garantizando la seguridad del acceso a la plataforma. Para el diseño de la interfaz se ha usado la aplicación Figma para crear un prototipo previo.

ABSTRACT

Lectarium is a web platform centered around a virtual library, developed as an intermodular project for the Web Application Development higher level training course. The application allows users to register on the platform and access a catalog of digital books, request loans to read books for a period of twenty days, mark favorites, and view both their loan history and all their favorites from a user dashboard. All of this is available without time or location restrictions.

The project's architecture follows the client-server pattern studied in the program. The frontend is developed using HTML5, CSS3, and JavaScript; the backend with Node.js and Express.js; and the database with MySQL. User authentication is implemented using passwords hashed with bcrypt and JWT tokens, ensuring secure access to the platform. Figma was used to design the interface and create a prior prototype.

JUSTIFICACIÓN DEL PROYECTO

Motivación principal:

La era digital ha transformado la manera en que consumimos contenido de entretenimiento, y el acceso a la cultura y al conocimiento a través de internet se ha convertido en una necesidad cotidiana. Y, aunque las bibliotecas físicas siguen siendo un recurso muy valioso, presentan ciertas limitaciones evidentes como horarios más restringidos, desplazamientos e incluso la disponibilidad limitada de ejemplares.

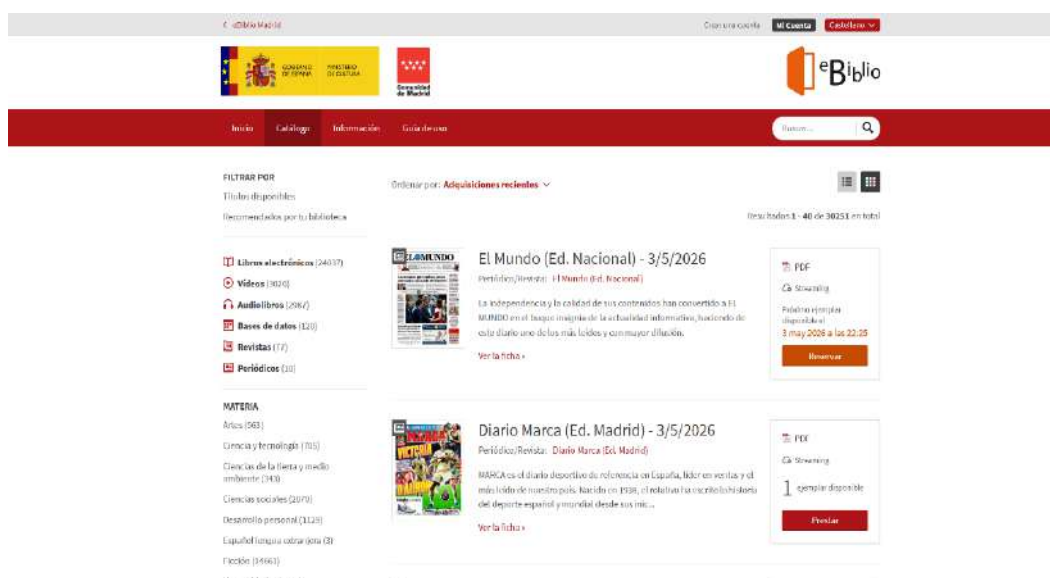
Por ello, nuestra idea “Lectarium” nace como una plataforma web enfocada en una biblioteca virtual que permite a los usuarios acceder a un catálogo de libros digitales, gestionar sus préstamos y leer desde cualquier dispositivo con conexión a internet, sin restricciones de horario ni ubicación.

Por otro lado, el proyecto supone una oportunidad para demostrar los conocimientos adquiridos a lo largo del ciclo formativo de Desarrollo de Aplicaciones Web, mediante su aplicación en un proyecto completo y totalmente real.

Estado de la cuestión:

Actualmente, existen varias plataformas enfocadas a la lectura de libros, archivos y documentos digitales, vamos a enumerar las más conocidas con una breve valoración de su interfaz y funcionalidades:

- **eBiblio:** Se trata de una plataforma pública de préstamo de libros digitales vinculada a bibliotecas municipales de España (Ministerio de cultura, s.f.). Permite a cualquier ciudadano con carné de biblioteca acceder gratuitamente a miles de títulos en formato digital. Su interfaz es funcional, aunque algo anticuada y no muy intuitiva. Se podría decir que es la plataforma más similar en funcionalidades a nuestra idea de proyecto. Por otro lado, su proceso de registro puede resultar complicado para usuarios no familiarizados con la tecnología y la aplicación móvil tiene valoraciones negativas.



- **Scribd/ KoboBooks / Kindle Unlimited:** Esto son plataformas comerciales con amplios catálogos de libros, pero se tratan de un servicio de suscripción con un coste mensual o anual. Ninguna de estas plataformas ofrece acceso gratuito ni un modelo de préstamo temporal como el de las bibliotecas tradicionales.
- **Project Gutenberg:** Se trata de una biblioteca gratuita de dominio público, pero sin sistema de préstamos ni gestión de usuarios (Hart M., s.f.). Su principal limitación es que su catálogo se restringe exclusivamente a obras cuyo copyright ha expirado, lo que excluye prácticamente a toda la biblioteca contemporánea. Además, no dispone de gestión de usuarios ni de préstamos, siendo simplemente un repositorio de descarga directa.

Tabla comparativa de plataformas

Plataforma	Gratuita	Gestión de usuarios	Sistema de préstamos	Interfaz moderna
eBiblio	Sí	Sí	Sí	No
Kindle Unlimi.	No	Sí	No	Sí
Scribd	No	Sí	No	Sí
P. Gutenberg	Sí	No	No	No
Lectarium	Sí	Sí	Sí	Sí

Conociendo las plataformas que existen actualmente en cuanto a libros digitales, hemos pensado en Lectarium como una combinación de gratuidad y accesibilidad a bibliotecas

públicas, añadiendo una interfaz moderna, intuitiva y responsive, mejorando en gran medida la experiencia de usuario.

Público objetivo:

Nuestra plataforma está dirigida a cualquier persona interesada en la lectura digital que busque una alternativa cómoda, gratuita y accesible a las bibliotecas físicas, especialmente usuarios habituados al entorno digital. De forma más específica, los perfiles de usuario serían los siguientes:

- Estudiantes de todos los niveles educativos que necesitan acceder a libros de literatura recomendada o material de estudio complementario.
- Lectores habituales que quieran disfrutar de una biblioteca digital sin restricciones de horario ni desplazamientos.
- Personas con movilidad reducida o que viven en zonas alejadas a bibliotecas físicas.
- Usuarios que prefieren consumir contenido en formato digital desde sus dispositivos habituales como ordenadores, tabletas o teléfonos móviles.
- Instituciones educativas que quieran ofrecer a sus alumnos acceso a recursos digitales de forma organizada.

INTRODUCCIÓN

Lectarium es una plataforma web de biblioteca virtual desarrollada en su totalidad con tecnologías web estándar. Su principal objetivo es ofrecer a los usuarios registrados acceso a un catálogo de libros digitales con sistema de préstamos temporales de veinte días, desde cualquier dispositivo y sin restricciones de horario ni de ubicación.

La aplicación está dividida en dos partes claramente diferenciadas: el frontend, que constituye la interfaz visible con la que interactúa el usuario, y el backend, que se encarga de gestionar la lógica del proyecto, la autenticación y la comunicación con la base de datos. Ambas partes se comunican a través de una API REST desarrollada con Node.js y Express.js. Finalmente, el proyecto ha sido desplegado mediante el uso de plataformas gratuitas, Railway.app para la parte del backend y Netlify para la parte de frontend.

El proyecto nace de la idea de mejorar la experiencia de usuario de las plataformas de biblioteca digital existentes, especialmente las de carácter público como eBiblio, que aunque es funcional hemos detectado en su uso que las interfaces son poco intuitivas y su diseño está desactualizado. Con Lectarium queremos proponer una alternativa moderna, accesible y visualmente atractiva que combina las ventajas de las bibliotecas públicas con la comodidad del entorno digital.

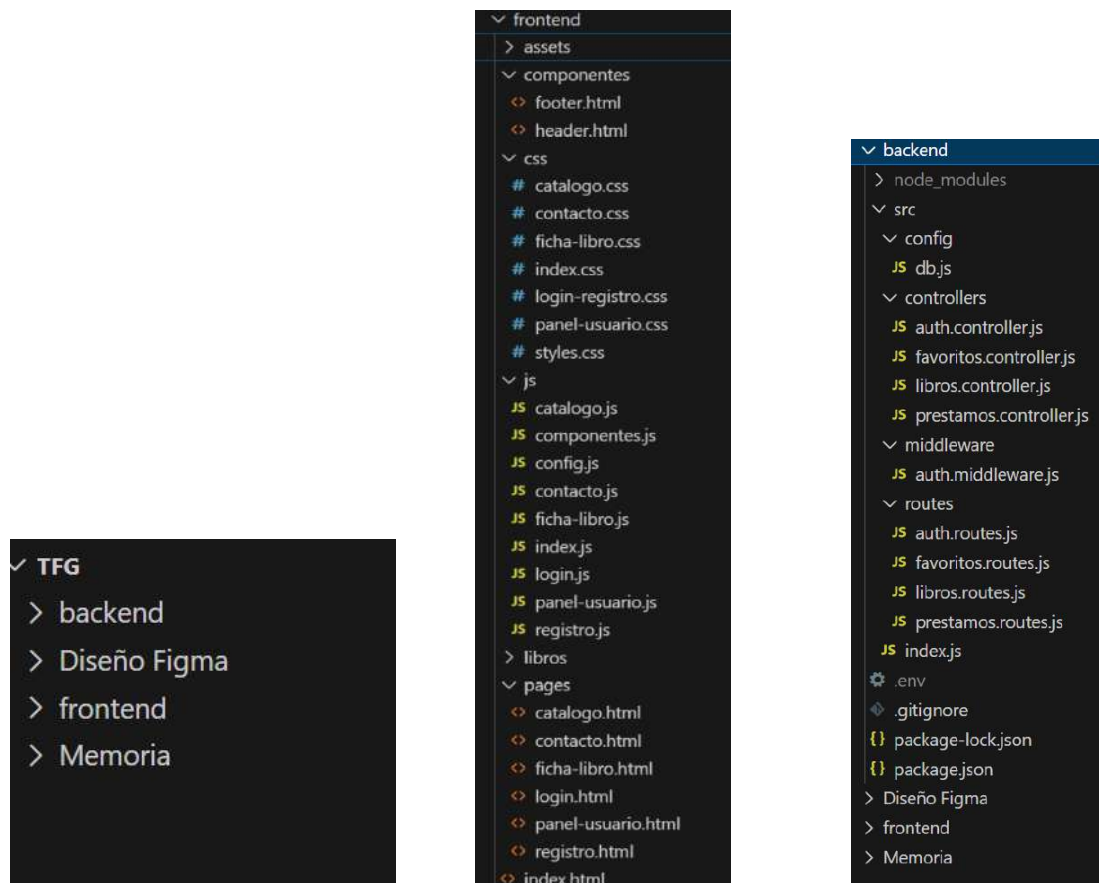
Las principales funcionalidades que resuelve son:

- Gestión de usuarios con registro e inicio de sesión seguros mediante contraseña hasheada y autenticación por token JWT (Okta, s.f.). El sistema diferencia entre usuarios con rol de usuario y rol de administrador.
- Acceso a un catálogo de libros digitales con búsqueda y filtrado por título, autor y género, así como una paginación para una navegación fluida entre los resultados.
- Página de inicio con carrusel de libros destacados, ordenados por número de veces que han sido marcados como favoritos, y sección de novedades con los últimos libros añadidos al catálogo.
- Ficha detallada de cada libro con información completa: portada, título, autor, sinopsis, disponibilidad en tiempo real y botones para solicitar préstamo y añadir a favoritos. Una vez solicitado el préstamo, el usuario podrá pulsar el botón de “Leer libro” para acceder al archivo del libro y comenzar a leerlo.

PROMETEO

- Sistema de préstamos digitales temporales con control de disponibilidad y expiración automática, límite de veinte días por préstamo y seguimiento del préstamo desde el panel de usuario.
- Gestión de favoritos para que el usuario pueda guardar los libros que más le interesen.
- Panel de usuario donde consultar el historial de préstamos realizado, diferenciando activos y vencidos; y los libros que el usuario ha marcado como favoritos. Además cuenta con la opción de cerrar sesión.
- Diseño responsive adaptado a dispositivos móviles, tabletas y ordenadores de escritorio.

El proyecto está organizado en una estructura de carpetas separando la parte de frontend de la de backend. Las siguientes imágenes muestran la estructura de carpetas vista desde el entorno de desarrollo utilizado para el desarrollo de la aplicación, Visual Studio Code.



OBJETIVOS

Los objetivos principales que se plantean resolver con el desarrollo de Lectarium son los siguientes:

1. Diseñar e implementar una base de datos relacional que refleje el catálogo de libros, los usuarios y sus interacciones, garantizando la integridad de los datos mediante el uso de claves primarias, foráneas y restricciones.
2. Desarrollar una API REST con Node.js y Express.js que gestione la lógica de negocio de la aplicación, incluyendo la autenticación, la gestión de libros, los préstamos y los favoritos.
3. Implementar un sistema de autenticación seguro con registro e inicio de sesión mediante correo electrónico y contraseña, utilizando bcrypt para el hash de contraseñas y JWT para la gestión de sesiones.
4. Diseñar y desarrollar una interfaz web intuitiva y responsive accesible desde cualquier dispositivo, siguiendo el prototipo diseñado en Figma.
5. Implementar un sistema de búsqueda y filtrado de libros dentro del catálogo en base a diferentes criterios como el título, el autor o autores y el género.
6. Implementar un sistema de préstamos digitales con control de disponibilidad, límite de tiempo y expiración automática.
7. Desplegar la aplicación en un entorno de producción accesible públicamente a través de internet.
8. Implementar un sistema de favoritos que permita al usuario guardar los libros de su interés.

Objetivos opcionales:

9. Implementar un sistema de valoraciones mediante una puntuación de 1 a 5 estrellas con comentario opcional.

RFTP

R01 – El sistema debe permitir el acceso únicamente a usuarios registrados.

R01F01 – El usuario debe poder registrarse en el sistema.

R01F01T01 – Crear la tabla “usuarios” en la base de datos con los campos: id_usuario, nombre, email, hash_contraseña, rol y fecha_registro.

PROMETEO

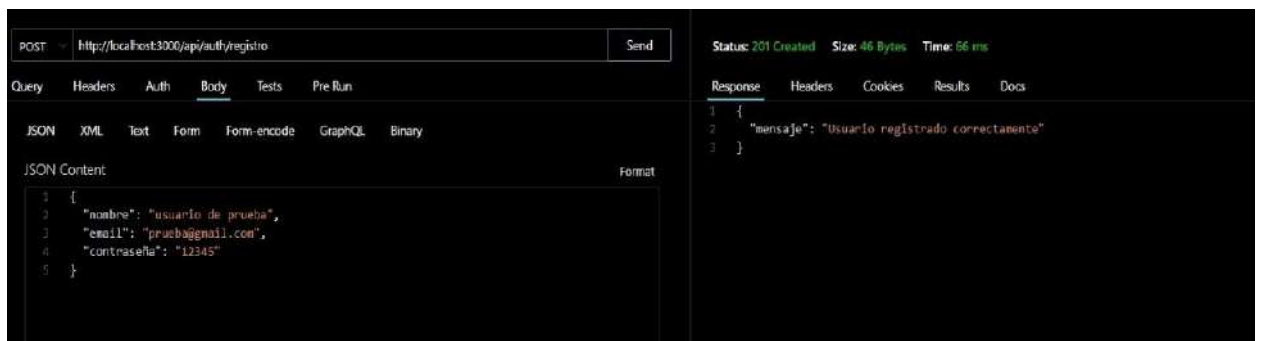
```
CREATE TABLE usuarios (  
  id_usuario INT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  email VARCHAR(150) NOT NULL,  
  hash_contraseña VARCHAR(150) NOT NULL UNIQUE,  
  rol ENUM('usuario', 'admin') NOT NULL DEFAULT 'usuario',  
  fecha_registro DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

R01F01T01P01 – Insertar un usuario de prueba y verificar que se almacena correctamente en la base de datos.

```
60  
61 • INSERT INTO usuarios (nombre, email, hash_contraseña, rol) VALUES  
62     ('Admin Lectarium', 'degregorio.rodriguez.andrea@gmail.com', '5f4dcc3b5aa765d61d8327deb882cf99', 'admin'),  
63     ('Lorena Gómez', 'lorena.gomez@email.com', '1jams11223934098asdnaksd87342938', 'usuario'),  
64     ('Francisco Toledo', 'francisco.toledo@email.com', 'asnd194b34562jvhv245hgd24f5432kjh', 'usuario'),  
65     ('Vanesa Pérez', 'vanesa.perez@email.com', '2k346f73kh65gf2j5gh427354kjh723j', 'usuario');  
66
```

R01F01T02 – Crear el endpoint POST/api/auth/registro en el backend que reciba nombre, email y contraseña, hashee la contraseña con bcrypt y la almacene en la base de datos.

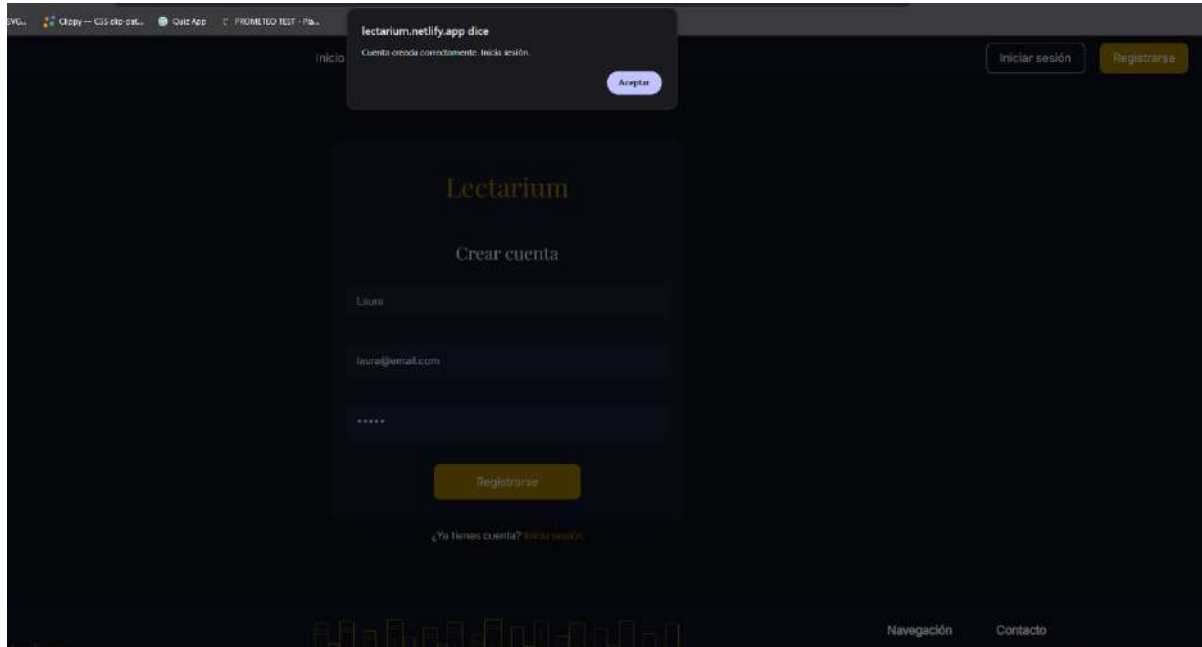
R01F01T02P01 – Enviar una petición de registro mediante el uso de la extensión Thunder Client para VSCode y verificar que devuelve el código HTTP 201 y el mensaje de confirmación.



R01F01T03 – Diseñar y desarrollar la pantalla `registro.html` con formulario de nombre, correo electrónico y contraseña, conectado al endpoint de registro.

PROMETEO

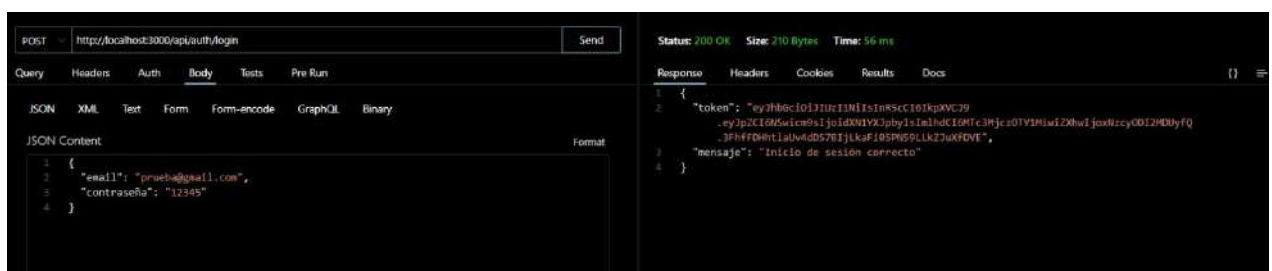
R01F01T03P01 – Visualizar la pantalla de registro y comprobar que el formulario envía los datos correctamente al backend.



R01F02 — El usuario debe poder iniciar sesión con su correo electrónico y contraseña.

R01F02T01 — Desarrollar el endpoint POST/api/auth/login que verifique las credenciales del usuario comparando la contraseña con el hash almacenado mediante `bcrypt.compare()` y que devuelva un token JWT firmado con la clave secreta.

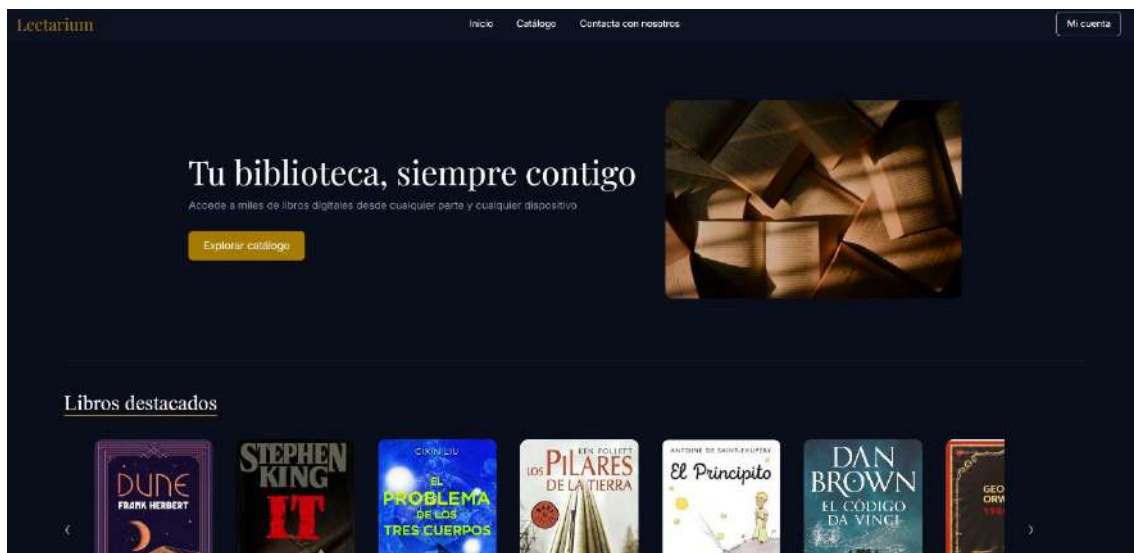
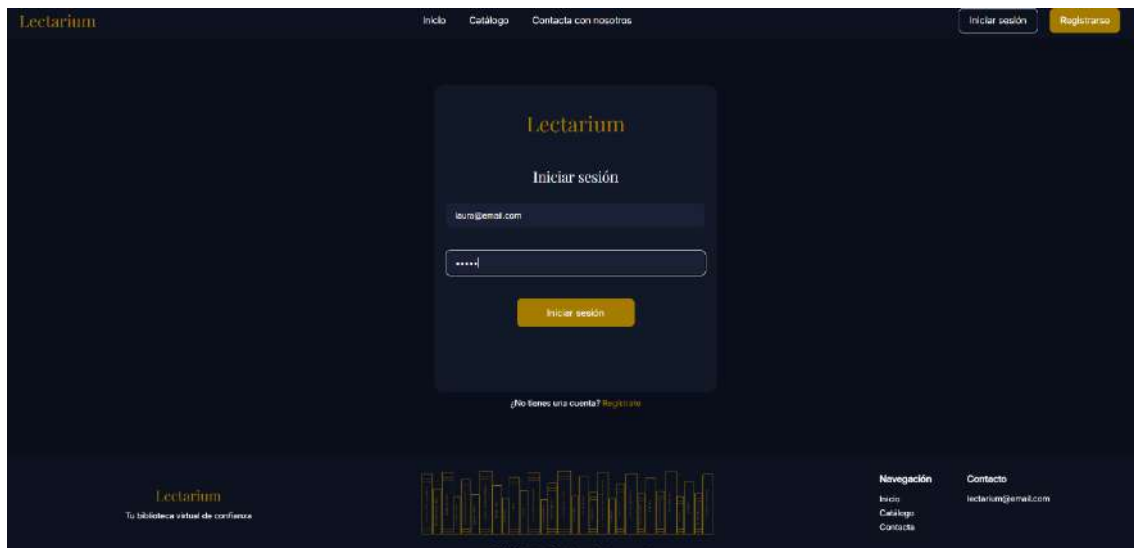
R01F02T01P01 — Enviar una petición POST de login desde Thunder Client y verificar que devuelve un token JWT válido con código HTTP 200.



PROMETEO

R01F02T02 — Diseñar y desarrollar la pantalla login.html con formulario de correo electrónico y contraseña, conectado al endpoint de login, que almacene el token en localStorage y redirija al inicio.

R01F02T02P01 — Visualizar la pantalla de inicio de sesión en el navegador, introducir credenciales correctas y comprobar que redirige correctamente a la página de inicio y el header cambia y muestra el botón “Mi Cuenta”.



R01F03 — El sistema debe proteger las rutas privadas mediante middleware de autenticación

PROMETEO

R01F03T01 — Desarrollar el middleware verificarToken que compruebe la presencia y validez del token JWT en la cabecera authorization de cada petición a rutas protegidas

R01F03T01P01 — Enviar una petición GET a una ruta protegida sin token desde Thunder Client y verificar que devuelve el código HTTP 401 con el mensaje de error correspondiente

R02 — El sistema debe ofrecer un catálogo de libros con búsqueda y filtrado.

R02F01 — El usuario debe poder ver el catálogo completo de libros.

R02F01T01 — Crear la tabla libros y géneros en la base de datos con sus respectivos campos y relaciones mediante clave foránea

```
• CREATE TABLE generos (  
  id_genero INT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL UNIQUE  
);  
  
• CREATE TABLE libros (  
  id_libro INT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
  titulo VARCHAR(250) NOT NULL,  
  autor VARCHAR(500) NOT NULL,  
  fecha_publicacion DATETIME NOT NULL,  
  ruta_archivo VARCHAR(500) NOT NULL,  
  disponibilidad TINYINT(1) NOT NULL DEFAULT 1,  
  id_genero INT NOT NULL,  
  CONSTRAINT fk_libro_genero FOREIGN KEY (id_genero) REFERENCES generos(id_genero)  
);
```

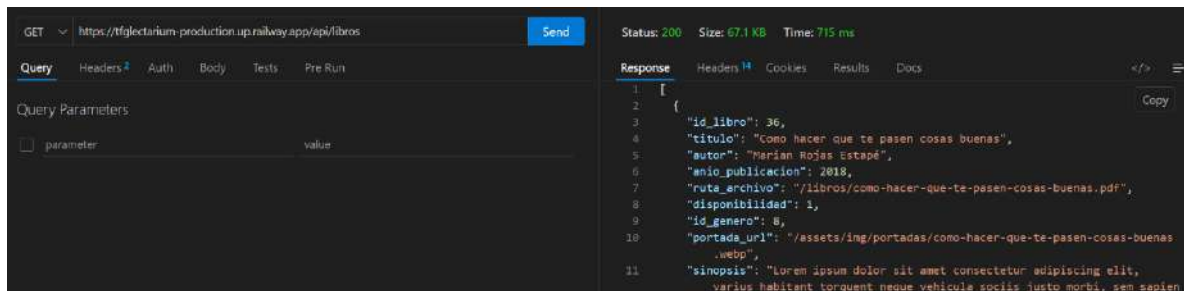
R02F01T01P01 — Insertar libros de prueba con diferentes géneros y verificar que se almacenan correctamente con sus relaciones

```
INSERT INTO libros (titulo, autor, anio_publicacion, id_genero, ruta_archivo) VALUES  
( 'El Código da Vinci', 'Dan Brown', 2003, 7, '/libros/codigo-davinci.pdf'),  
( 'It', 'Stephen King', 1986, 2, '/libros/it.pdf'),  
( 'Dune', 'Frank Herbert', 1965, 3, '/libros/dune.pdf'),  
( 'Los Pilares de la Tierra', 'Kent Follet', 1989, 9, '/libros/codigo-davinci.pdf'),  
( 'El Principito', 'Antoine de Saint-Exupéry', 1943, 5, '/libros/el-principito.pdf');
```

PROMETEO

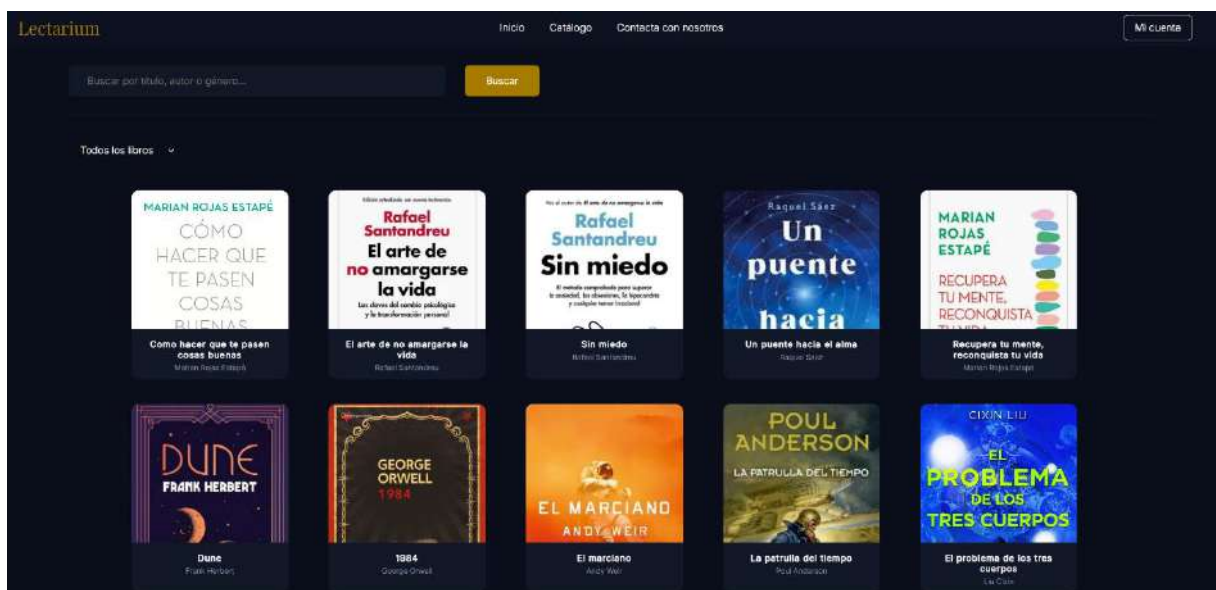
R02F01T02 — Desarrollar el endpoint GET/api/libros que devuelva todos los libros del catálogo con su género asociado mediante JOIN entre las tablas libros y géneros.

R02F01T02P01 — Enviar una petición GET desde Thunder Client y verificar que devuelve la lista completa de libros con el nombre del género incluido.



R02F01T03 — Diseñar y desarrollar la pantalla catalogo.html con grid de tarjetas de libros cargadas dinámicamente desde la API, con paginación de 15 libros por página.

R02F01T03P01 — Visualizar el catálogo y comprobar que se muestran los libros correctamente con sus portadas y autores.



PROMETEO

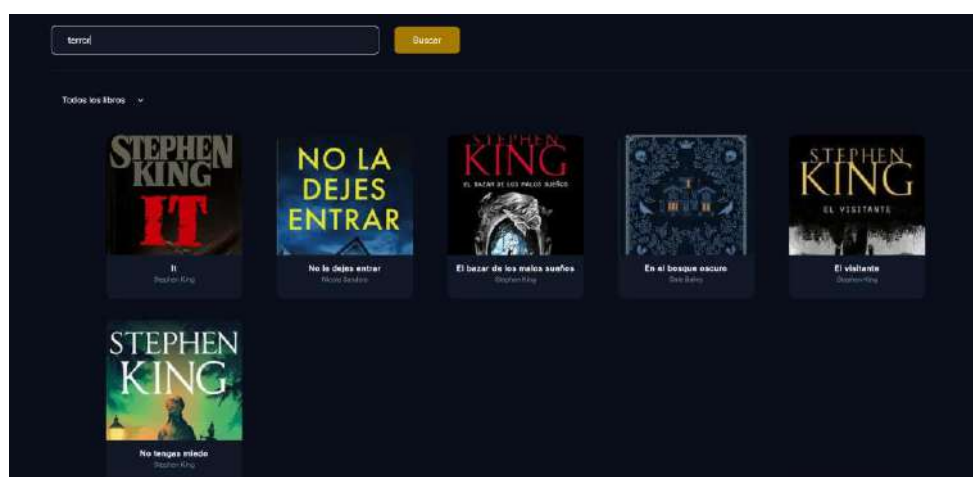
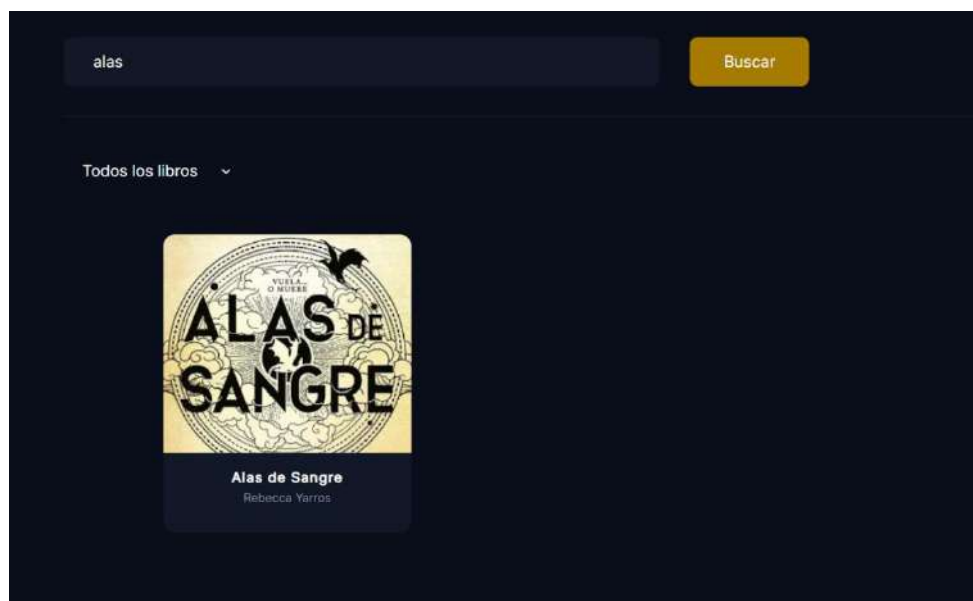
R02F02 — El usuario debe poder buscar libros por título, autor o género.

R02F02T01 — Desarrollar el endpoint GET/api/libros/buscar?query= con búsqueda mediante LIKE en los campos de título, autor y género.

R02F02T01P01 — Buscar un término concreto y verificar que devuelve únicamente los libros que contienen ese término en alguno de los tres campos.

R02F02T02 — Implementar el buscador en catalogo.html conectado al endpoint de búsqueda.

R02F02T02P01 — Introducir un término en el buscador y comprobar que el grid se actualiza correctamente.



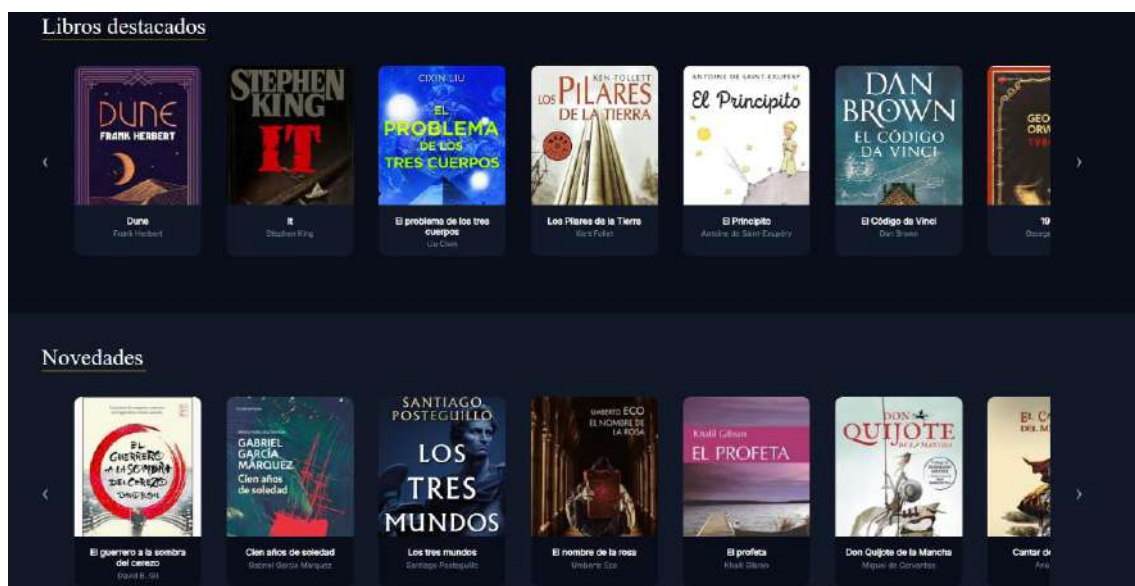
R02F03 — El sistema debe mostrar libros destacados y novedades en la página de inicio

R02F03T01 — Desarrollar el endpoint GET /api/libros/destacados que devuelva los 10 libros con mayor número de veces marcados como favoritos, y el endpoint GET /api/libros/novedades que devuelva los 10 últimos libros añadidos al catálogo.

R02F03T01P01 — Verificar desde Thunder Client que ambos endpoints devuelven exactamente 10 libros ordenados correctamente.

R02F03T02 — Implementar los carruseles de libros destacados y novedades en index.html cargados dinámicamente desde la API

R02F03T02P01 — Visualizar la página de inicio y comprobar que los dos carruseles muestran libros diferentes con sus portadas correctas.



R03 — El sistema debe gestionar préstamos digitales temporales.

R03F01 — El usuario debe poder solicitar un préstamo de un libro disponible.

R03F01T01 — Crear la tabla préstamos en la base de datos con los campos id_prestamo, id_usuario, id_libro, fecha_inicio, fecha_fin y estado.

PROMETEO

```
CREATE TABLE prestamos (  
  id_prestamo INT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
  id_usuario INT NOT NULL,  
  id_libro INT NOT NULL,  
  fecha_inicio DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  fecha_fin DATETIME NOT NULL DEFAULT (CURRENT_TIMESTAMP + INTERVAL 20 DAY),  
  estado ENUM('activo', 'vencido') NOT NULL DEFAULT 'activo',  
  CONSTRAINT fk_prestamo_usuario FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario),  
  CONSTRAINT fk_prestamo_libro FOREIGN KEY (id_libro) REFERENCES libros(id_libro)  
);
```

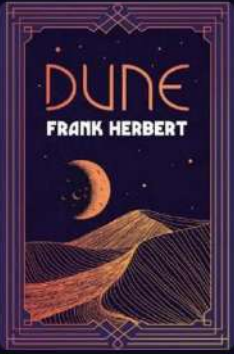
R03F01T01P01 — Insertar un préstamo de prueba y verificar que se almacena correctamente.

R03F01T02 — Desarrollar el endpoint POST /api/prestamos que compruebe la disponibilidad del libro, registre el préstamo y actualice el campo disponibilidad del libro a 0.

R03F01T02P01 — Solicitar un préstamo desde Thunder Client y verificar que devuelve código 201 y marca el libro como no disponible.

R03F01T03 — Implementar el botón Solicitar préstamo en ficha-libro.html conectado al endpoint de préstamos, que se deshabilite automáticamente cuando el libro no esté disponible.

R03F01T03P01 — Hacer click en el botón y comprobar que el préstamo se registra y el botón cambia a "No disponible".



Dune
FRANK HERBERT

Dune
Frank Herbert
Género: Ciencia ficción
Año de publicación: 1965
✓ Disponible

Sinopsis
Dune es la primera novela de la serie homónima «Dune» de Frank Herbert, una obra maestra unánimemente reconocida como la mejor saga de ciencia ficción de todos los tiempos. Arrakis: un planeta desértico donde el agua es el bien más preciado y, donde llorar a los muertos es el símbolo de máxima prodigalidad.

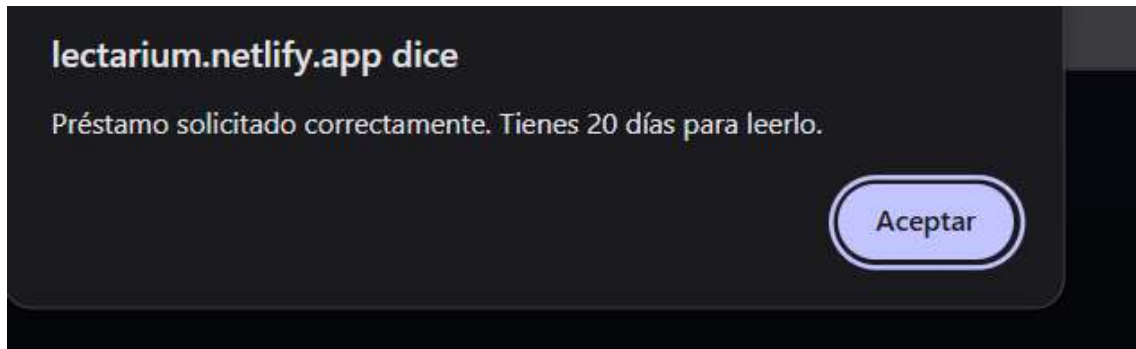
Paul Atreides: un adolescente marcado por un destino singular, dotado de extraños poderes y, abocado a convertirse en dictador, mesías y mártir.

Los Harkonnen: personificación de las intrigas que rodean el Imperio Galáctico, buscan obtener el control sobre Arrakis para disponer de la melange, preciosa especia y uno de los bienes más codiciados del universo.

Los Fremen: seres libres que han convertido el inhóspito paraje de Dune en su hogar, y que se sienten orgullosos de su pasado y temerosos de su futuro.

Solicitar préstamo

Añadir a favoritos



R03F02 — El usuario debe poder consultar su historial de préstamos.

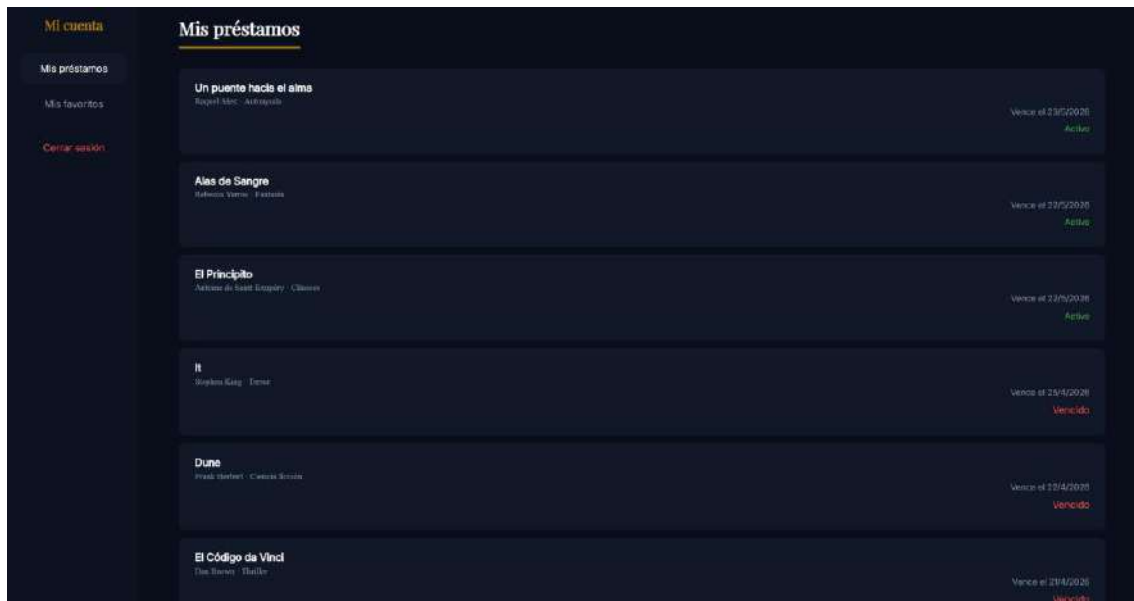
R03F02T01 — Desarrollar el endpoint GET /api/prestamos/historial que devuelva los préstamos del usuario autenticado con información del libro y fecha de vencimiento.

R03F02T01P01 — Enviar una petición GET con token desde Thunder Client y verificar que devuelve los préstamos correctamente.

R03F02T02 — Implementar la sección "Mis préstamos" en panel-usuario.html con tarjetas que muestren el título, autor, fecha de vencimiento y estado de cada préstamo.

R03F02T02P01 — Acceder al panel de usuario en el navegador y comprobar que se muestran correctamente los préstamos activos y el historial del usuario autenticado.

PROMETEO



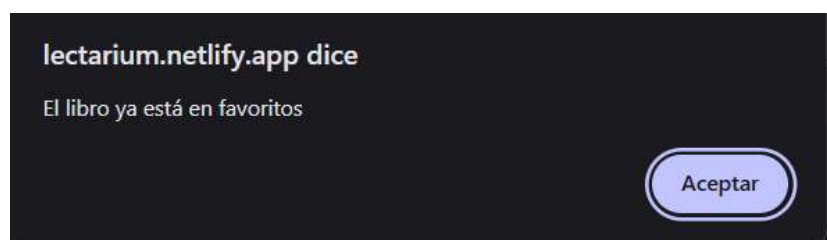
R04 — El sistema debe permitir al usuario marcar libros como favoritos y poder verlos en su panel de usuario

R04F01 — El usuario debe poder añadir libros a favoritos

R04F01T01 — Crear la tabla favoritos en la base de datos con clave primaria compuesta por id_usuario e id_libro.

```
CREATE TABLE favoritos (  
  id_usuario INT NOT NULL,  
  id_libro INT NOT NULL,  
  PRIMARY KEY (id_usuario, id_libro),  
  CONSTRAINT fk_fav_usuario FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario),  
  CONSTRAINT fk_fav_libro FOREIGN KEY (id_libro) REFERENCES libros(id_libro)  
);
```

R04F01T01P01 — Verificar que al intentar insertar el mismo favorito dos veces se produce el error ER_DUP_ENTRY y se devuelve el mensaje de error apropiado.



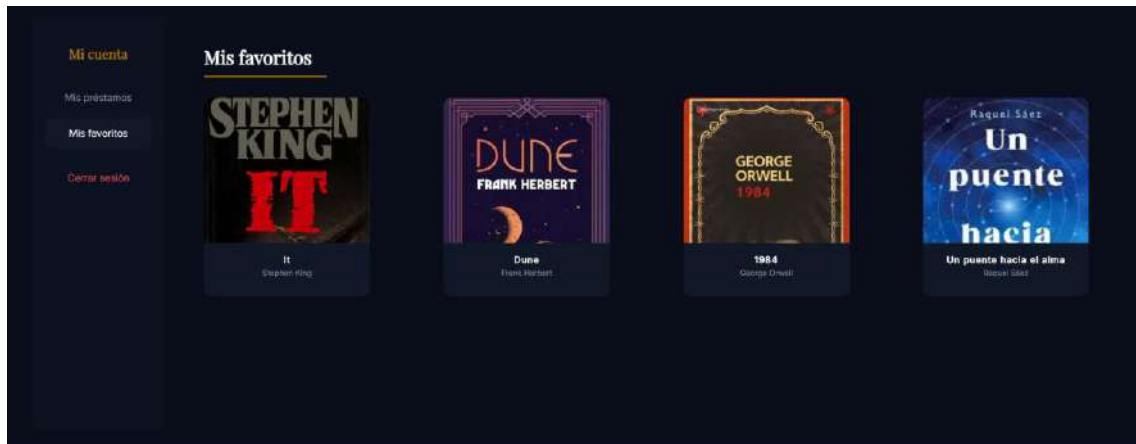
PROMETEO

R04F01T02 — Desarrollar los endpoints POST /api/favoritos para añadir, GET /api/favoritos para listarlos.

R04F01T02P01 — Probar desde Thunder Client con token válido y verificar que funcione correctamente.

R04F01T03 — Implementar el botón de favoritos en ficha-libro.html y la sección Mis favoritos en panel-usuario.html con grid de tarjetas de libros.

R04F01T03P01 — Añadir un libro a favoritos desde la ficha y comprobar que aparece correctamente en la sección Mis favoritos del panel de usuario.



DESCRIPCIÓN

Arquitectura

Nuestro proyecto Lectarium sigue una arquitectura cliente-servidor de tres capas completamente desacopladas que se comunican a través de una API REST. Esta arquitectura es la más extendida en el desarrollo web moderno porque permite desarrollar, mantener y escalar cada capa de forma independiente:

- Capa frontend: Constituye la interfaz visible con la que interactúa el usuario. Está desarrollada con HTML5, CSS3 y JavaScript vanilla, sin dependencia de ningún framework de frontend. Se comunica con el backend exclusivamente mediante la API `fetch()` de JavaScript, realizando peticiones HTTP y procesando las respuestas en formato JSON para actualizar el DOM dinámicamente sin recargar la página.
- Capa backend: Es el núcleo de la aplicación. Está desarrollado con Node.js y Express.js y expone una API REST con endpoints bien definidos para cada operación. Gestiona la autenticación mediante JWT, aplica el middleware de verificación de token en las rutas protegidas, valida los datos de entrada y coordina la comunicación con la base de datos a través del conector `mysql2`.
- Capa de base de datos: Almacena toda la información persistente de la aplicación en un servidor MySQL. La base de datos está estructurada en cinco tablas relacionadas: usuarios, libros, generos, prestamos y favoritos, con sus correspondientes claves primarias, foráneas y restricciones de integridad.

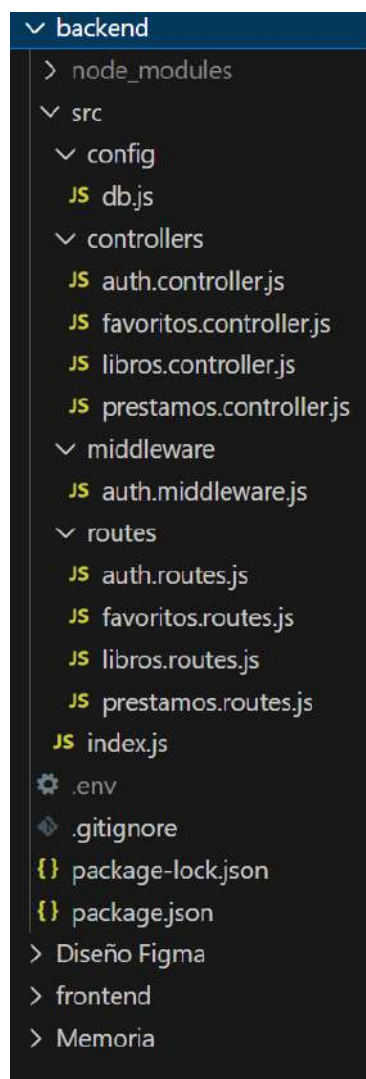
El flujo de una petición sería el siguiente:



Estructura de carpetas del proyecto

Backend → /backend

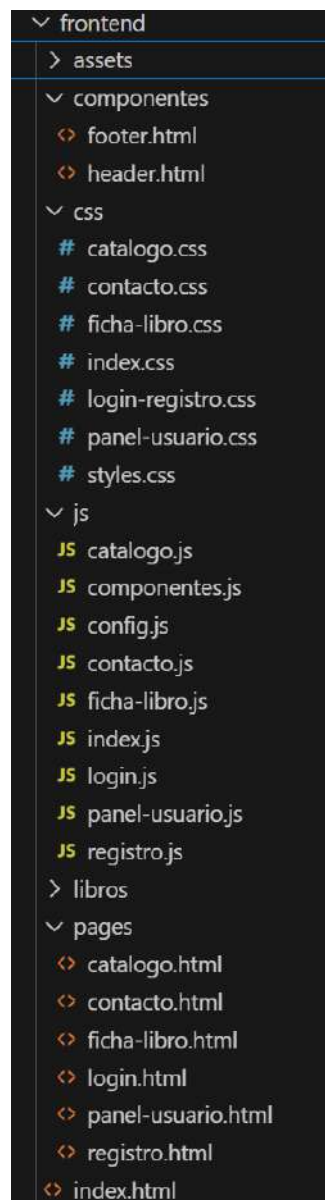
- src/config/db.js — configuración de la conexión a MySQL
- src/controllers/ — lógica de cada entidad: auth, libros, préstamos, favoritos
- src/middleware/auth.middleware.js — verificación del token JWT
- src/routes/ — definición de rutas de la API
- src/index.js — punto de entrada del servidor
- .env — variables de entorno con credenciales



PROMETEO

Frontend —> /frontend

- index.html — página de inicio con carruseles
- pages/ — páginas secundarias: catálogo, ficha, login, registro, panel
- css/ — archivos de estilos por página más estilos globales
- js/ — archivos JavaScript por página más componentes reutilizables
- componentes/ — header.html y footer.html para carga dinámica
- assets/img/ — imágenes y portadas de libros
- assets/fonts/ — fuentes de texto implementadas en CSS



PRINCIPALES CASOS DE USO

1. Iniciar sesión

DESCRIPCIÓN	El usuario introduce sus datos personales para crear una cuenta nueva en la plataforma
PRECONDICIONES	El usuario no debe tener una cuenta registrada con ese email
POSTCONDICIONES	Se crea un nuevo registro en la tabla usuarios con la contraseña hasheada y el rol de usuario por defecto. Se redirige al login.
DATOS DE ENTRADA	nombre, email, contraseña
DATOS DE SALIDA	Mensaje de confirmación, redirección a login.html
TABLAS	Usuarios
INTERFACES	registro.html

2. Iniciar sesión

DESCRIPCIÓN	El usuario introduce su email y contraseña para acceder a la plataforma
PRECONDICIONES	El usuario debe estar registrado previamente en el sistema
POSTCONDICIONES	Se genera un token JWT y el usuario obtiene acceso al catálogo de libros
DATOS DE ENTRADA	Email y contraseña
DATOS DE SALIDA	Token JWT y mensaje de confirmación
TABLAS	usuarios
INTERFACES	login.html

3. Buscar libro

DESCRIPCIÓN	El usuario introduce un término de búsqueda para encontrar libros en el catálogo
PRECONDICIONES	Ninguna. La búsqueda es accesible sin iniciar sesión.
POSTCONDICIONES	El grid del catálogo se actualiza mostrando únicamente los libros que coinciden con el término buscado en título, autor o género.
DATOS DE ENTRADA	query (texto de búsqueda), filtro de disponibilidad
DATOS DE SALIDA	Lista de libros filtrados en formato JSON
TABLAS	libros, generos
INTERFACES	catalogo.html

4. Solicitar préstamo

DESCRIPCIÓN	El usuario solicita el préstamo de un libro que figura como disponible en el catálogo
PRECONDICIONES	El usuario debe haber iniciado sesión y el libro debe aparecer como disponible
POSTCONDICIONES	Se registra el préstamo con fecha de vencimiento a 20 días y el libro se marca como No disponible
DATOS DE ENTRADA	Id_libro y token JWT
DATOS DE SALIDA	Mensaje de confirmación e id_prestamo
TABLAS	Préstamos y libros
INTERFACES	Ficha-libro.html

5. Gestionar favoritos

DESCRIPCIÓN	El usuario añade o consulta los libros marcados como favoritos
PRECONDICIONES	El usuario debe haber iniciado sesión
POSTCONDICIONES	El libro queda registrado en la tabla favoritos. El usuario puede verlo en la sección Mis favoritos de su panel.
DATOS DE ENTRADA	id_libro, token JWT
DATOS DE SALIDA	Mensaje de confirmación
TABLAS	favoritos, libros
INTERFACES	ficha-libro.html, panel-usuario.html

DISEÑOS

Diagrama Entidad-Relación

El diagrama Entidad-Relación (ER) representa el modelo conceptual de la base de datos de Lectarium. En él se muestran las entidades principales del sistema, sus atributos y las relaciones entre ellas, indicando la cardinalidad de cada relación.

Las entidades identificadas son USUARIOS, LIBROS, GÉNEROS, PRÉSTAMOS y FAVORITOS. Las relaciones principales son:

- USUARIOS — LIBROS a través de PRÉSTAMOS: relación N:M. Un usuario puede tener muchos préstamos y un libro puede haber sido tomado prestado por muchos usuarios.
- USUARIOS — LIBROS a través de FAVORITOS: relación N:M. Un usuario puede marcar muchos libros como favoritos y un libro puede ser marcado por muchos usuarios.
- LIBROS — GÉNEROS: relación N:1. Muchos libros pertenecen a un género, pero cada libro pertenece a exactamente un género.

PROMETEO

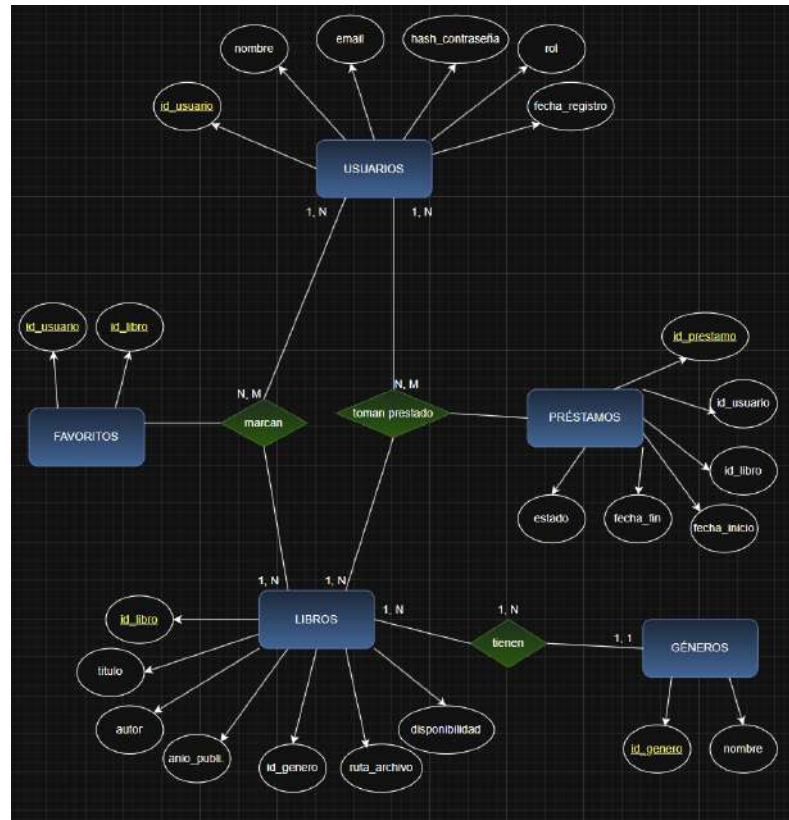


Diagrama de la base de datos

El diagrama relacional extendido muestra el modelo físico de la base de datos, con las tablas, sus columnas, tipos de datos, claves primarias y claves foráneas que implementan las relaciones definidas en el diagrama ER.

PROMETEO

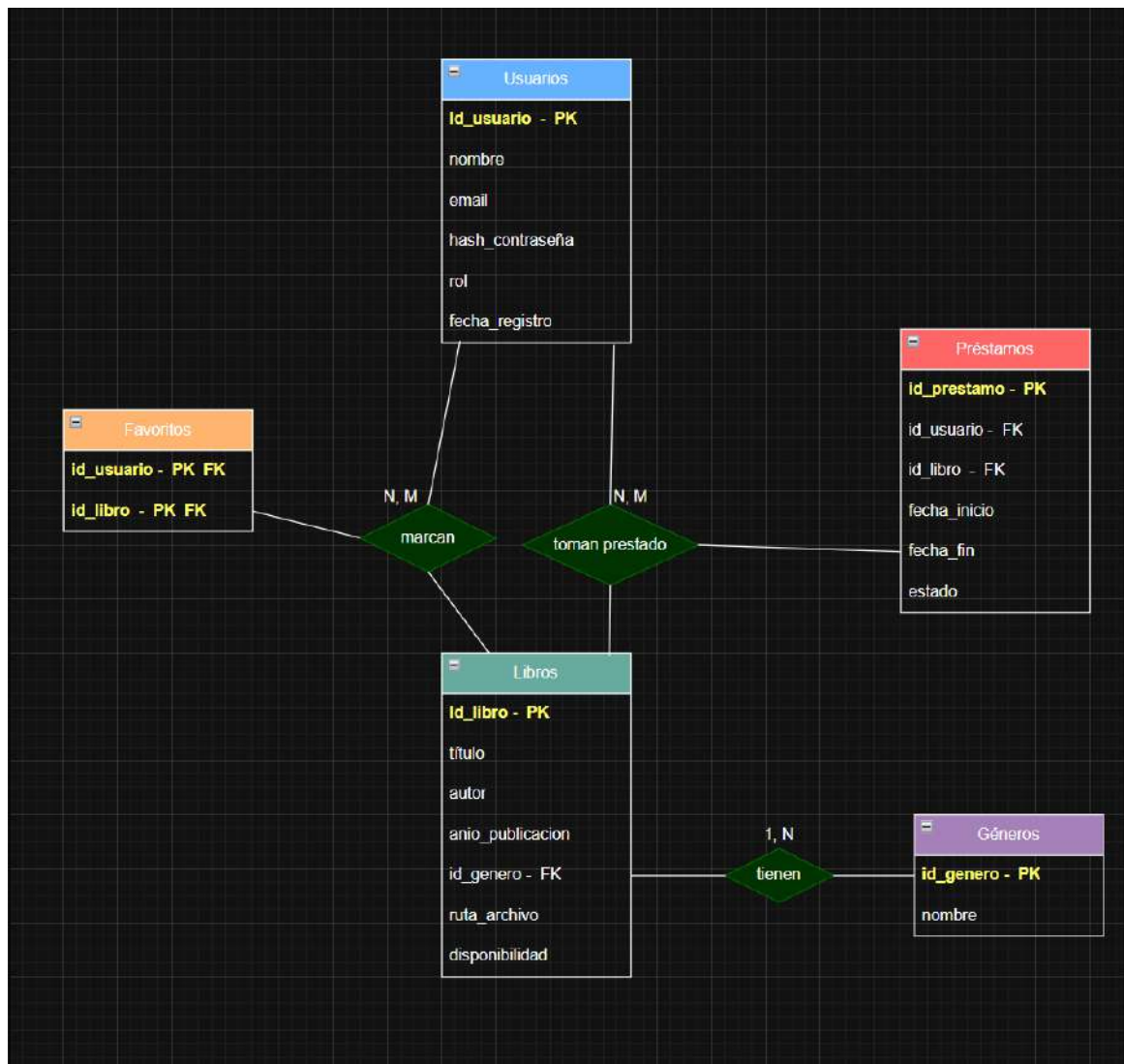


Diagrama extendido realizado mediante Draw.io

PROMETEO

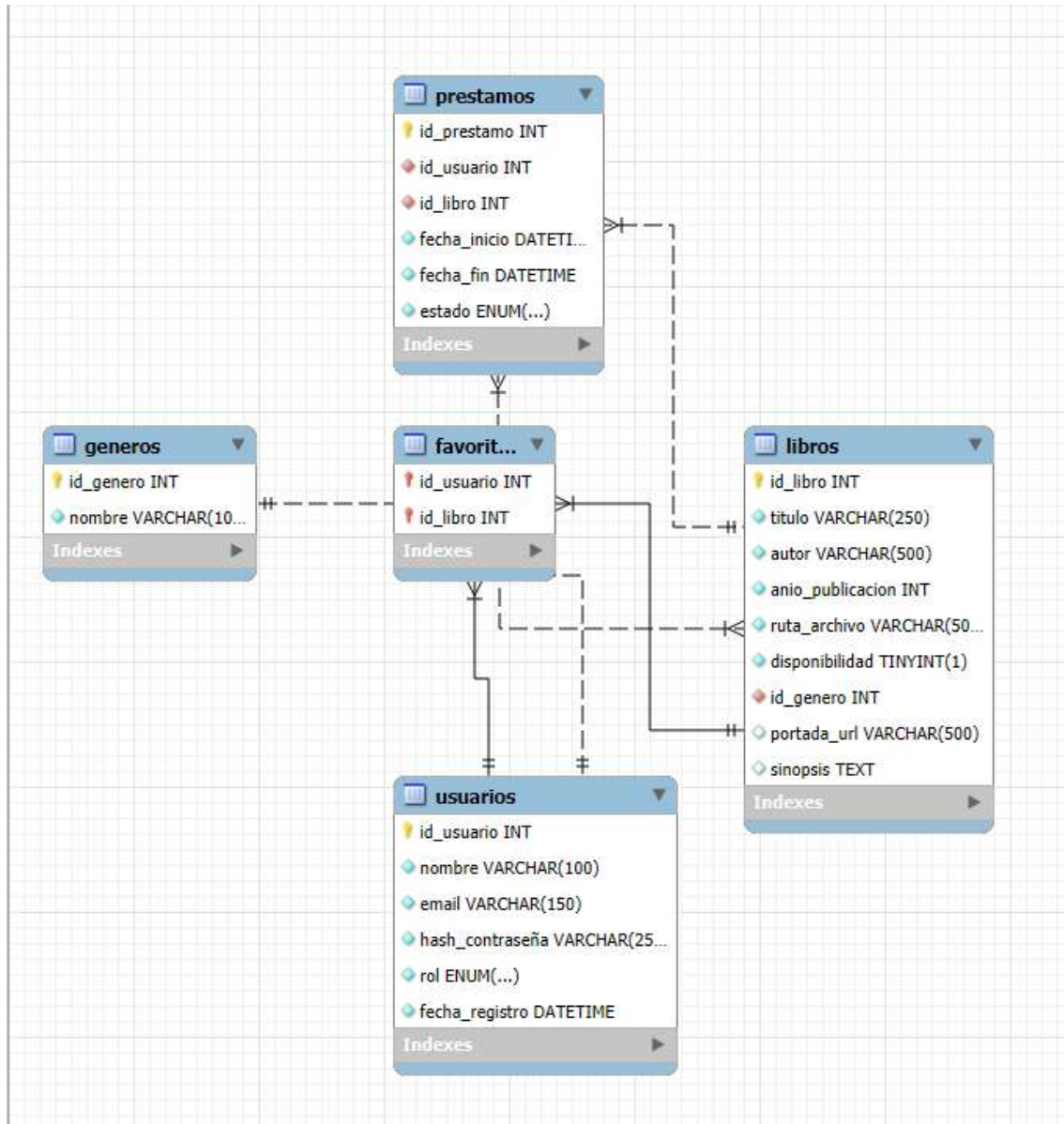


Diagrama relacional extraído de MySQL Workbench tras la elaboración de la base de datos.

A continuación se describe en detalle cada tabla de la base de datos:

Tabla: usuarios

Campo	Tipo	Restricciones	Descripción
id_usuario	INT	PK, AUTO_INCREMENT	Identificador único del usuario
Nombre	VARCHAR(100)	NOT NULL	Nombre completo del usuario

PROMETEO

Email	VARCHAR(150)	NOT NULL, UNIQUE	Correo electrónico del usuario
Hash_contraseña	VARCHAR(250)	NOT NULL	Contraseña hasheada con bcrypt
Rol	ENUM	DEFAULT 'usuario'	Rol del usuario: usuario o admin
Fecha_registro	DATETIME	DEFAULT NOW()	Fecha y hora de registro

Tabla: géneros

Campo	Tipo	Restricciones	Descripción
id_genero	INT	PK, AUTO_INCREMENT	Identificador único del género
Nombre	VARCHAR(100)	NOT NULL, UNIQUE	Nombre del género literario

Tabla: libros

Campo	Tipo	Restricciones	Descripción
id_libro	INT	PK, AUTO_INCREMENT	Identificador único del libro
Título	VARCHAR(255)	NOT NULL	Título del libro
Autor	VARCHAR(500)	NOT NULL	Autor o autores de la obra
Anio_publicacion	INT	NOT NULL	Año de publicación de la obra
Ruta_archivo	VARCHAR(500)	NOT NULL	Ruta al archivo digital del libro
Portada_url	VARCHAR(500)	DEFAULT default	Ruta a la imagen de la portada
Sinopsis	TEXT	NULL	Resumen de la obra
Disponibilidad	TINYINT(1)	DEFAULT 1	1: disponible, 0: prestado

PROMETEO

Id_genero	INT	FK → generos.id_genero	Género literario del libro
-----------	-----	---------------------------	----------------------------

Tabla: prestamos

Campo	Tipo	Restricciones	Descripción
id_prestamo	INT	PK, AUTO_INCREMENT	Identificador único del préstamo
Id_usuario	INT	FK → usuarios.id_usuario	Usuario que realiza el préstamo
Id_libro	INT	FK → libros.id_libro	Libro prestado
Fecha_inicio	DATETIME	DEFAULT NOW()	Fecha de inicio del préstamo
Fecha_fin	DATETIME	DEFAULT +20 días	Fecha de vencimiento del préstamo
estado	ENUM	DEFAULT 'activo'	Estado: activo o vencido

Tabla: favoritos

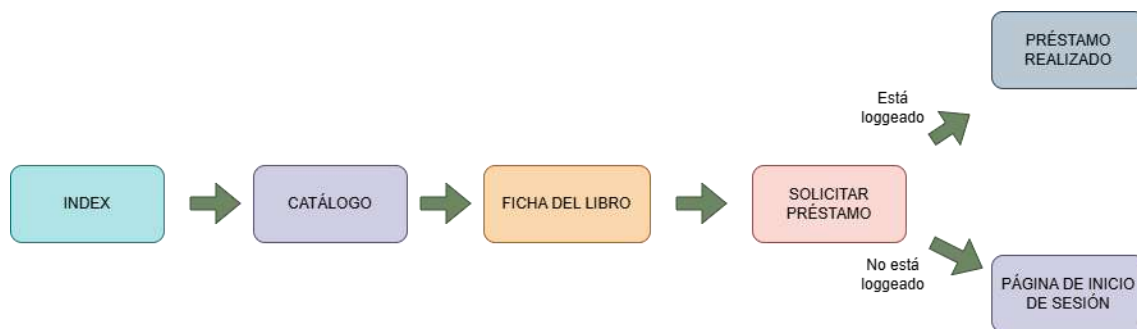
Campo	Tipo	Restricciones	Descripción
id_usuario	INT	PK, FK → usuarios.id_usuario	Usuario que marca el favorito
id_libro	INT	PK, FK → libros.id_libro	Libro marcado como favorito

Diagrama de flujo de navegación

El diagrama de flujo de navegación muestra cómo el usuario se mueve entre las distintas páginas de la aplicación y qué acciones desencadenan cada transición.

- El usuario accede a la página de inicio (index.html) donde visualiza los libros destacados y las novedades.
- Desde cualquier página puede navegar al catálogo (catalogo.html) para buscar libros.

- Al hacer click en una tarjeta accede a la ficha del libro (ficha-libro.html).
- Si intenta solicitar un préstamo o añadir a favoritos sin sesión, es redirigido al login (login.html).
- Desde el login puede navegar al registro (registro.html) si no tiene cuenta.
- Una vez autenticado, el header muestra el botón Mi cuenta que lleva al panel de usuario (panel-usuario.html).



Diseño de la interfaz en Figma

Antes de comenzar el desarrollo del frontend, se diseñó un prototipo completo de todas las pantallas de la aplicación en Figma. Este proceso permitió definir con precisión la paleta de colores, la tipografía, el layout de cada página y la experiencia de usuario antes de escribir una sola línea de código.

Paleta de colores:

Nombre	Código HEX	Uso
Fondo principal	#0A0E1A	Fondo de todas las páginas
Fondo secundario	#111827	Tarjetas, formularios, sidebar
Fondo navbar/footer	#0D1221	Header y footer
Acento dorado	#A67C00	Botones primarios, logo, detalles

PROMETEO

Texto principal	#FFFFFF	Títulos y texto en fondos oscuros
Texto secundario	#8892A4	Subtítulos, metadatos, placeholders
Estado disponible	#4CAF50	Indicador de libro disponible
Estado no disponible	#E05555	Indicador de libro no disponible

Tipografía utilizada:

- Playfair Display — fuente serif elegante utilizada para títulos principales, el logo y encabezados de sección. Aporta personalidad editorial y refinamiento al diseño.
- Inter — fuente sans-serif moderna y muy legible utilizada para el cuerpo del texto, la navegación, los botones y los metadatos. Garantiza una lectura cómoda en todos los tamaños.

Imágenes de las pantallas diseñadas:



Pantalla de inicio con una parte introductoria y dos secciones de libros destacados y novedades con carruseles y tarjetas.

PROMETEO

Registro

Lectarium Inicio Catálogo Sobre nosotros Iniciar sesión Registrarse

Lectarium

Crear cuenta

Nombre completo

Correo electrónico

Contraseña

Registrarse

¿Ya tienes cuenta? [Inicia sesión](#)

Login

Lectarium Inicio Catálogo Sobre nosotros Iniciar sesión Registrarse

Lectarium

Iniciar sesión

Correo electrónico

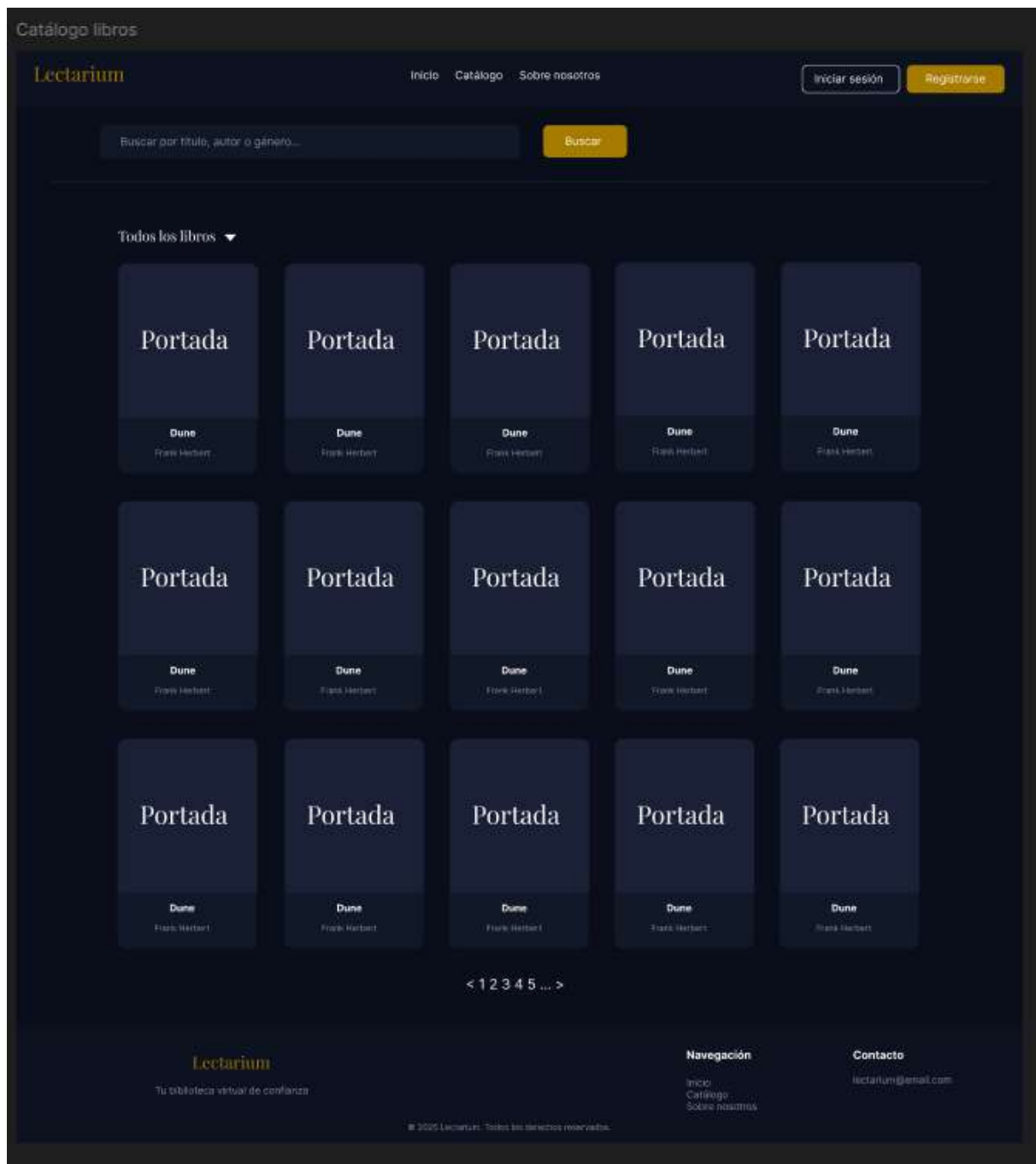
Contraseña

Iniciar sesión

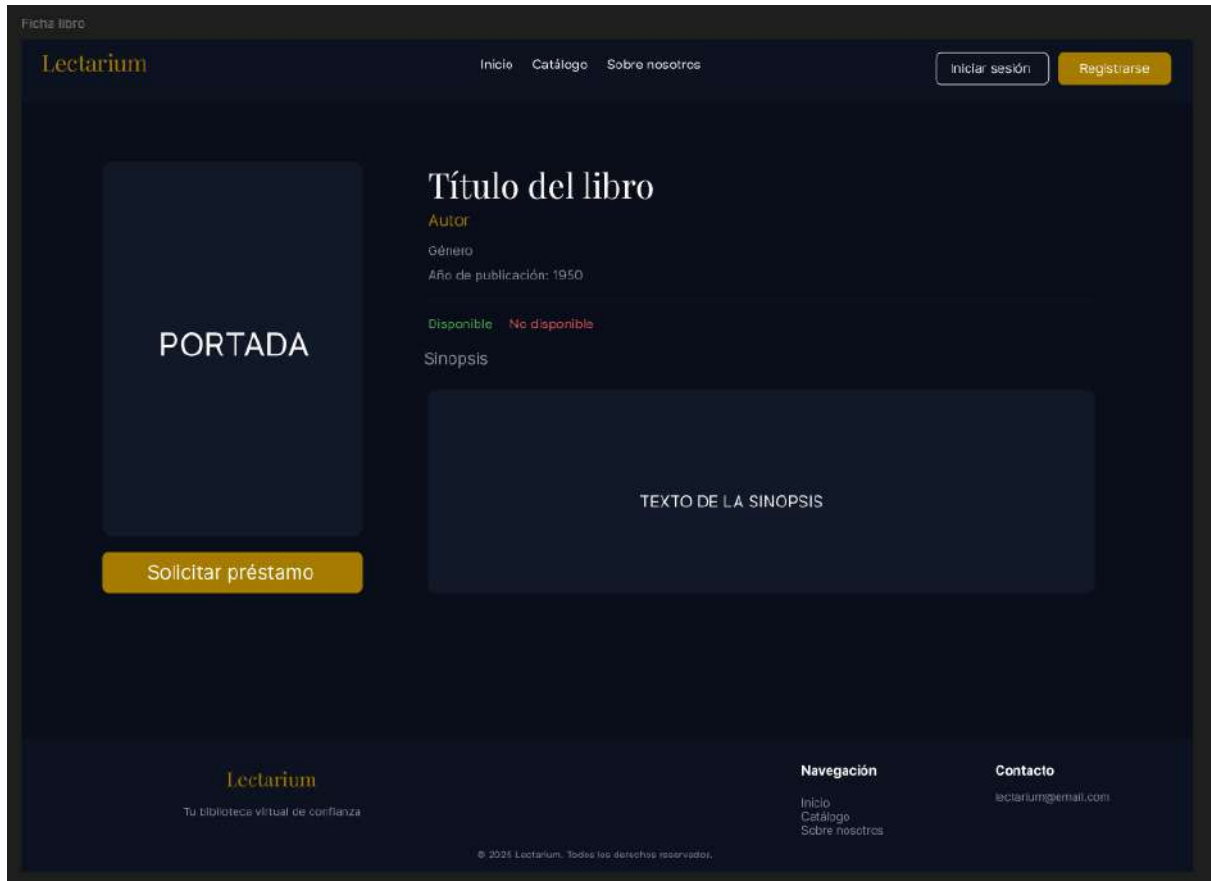
¿No tienes cuenta? [Regístrate](#)

Páginas de inicio de sesión y registro de usuarios con un formulario de acceso y registro.

PROMETEO



Página del catálogo de libros con barra de búsqueda, filtro de disponibilidad y paginación en la parte inferior.



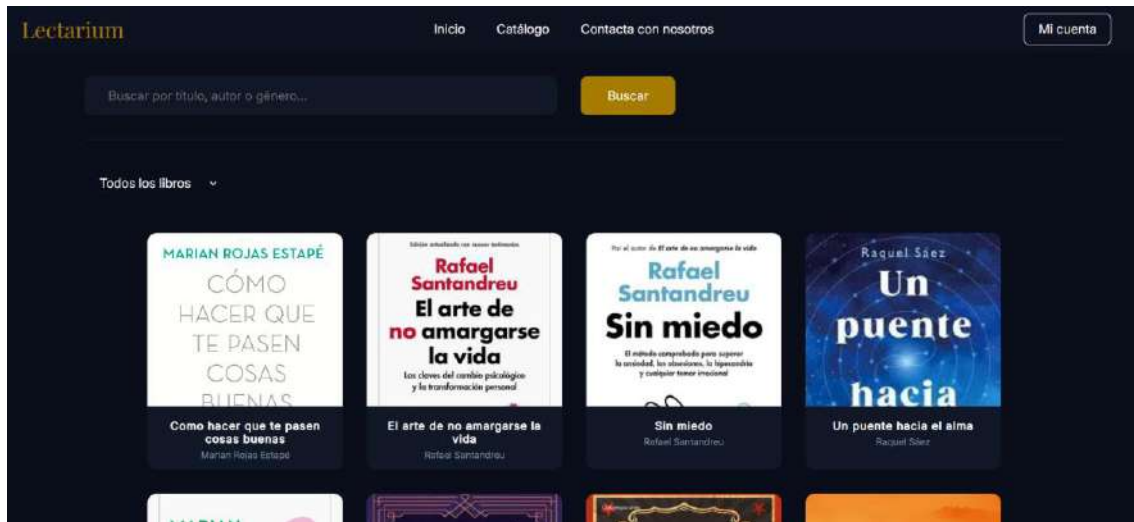
Página de la ficha de libro con la portada e información de cada libro, inicialmente solo con el botón de Solicitar préstamo, más tarde se añadiría el botón de favoritos.

Interfaces en distintos dispositivos

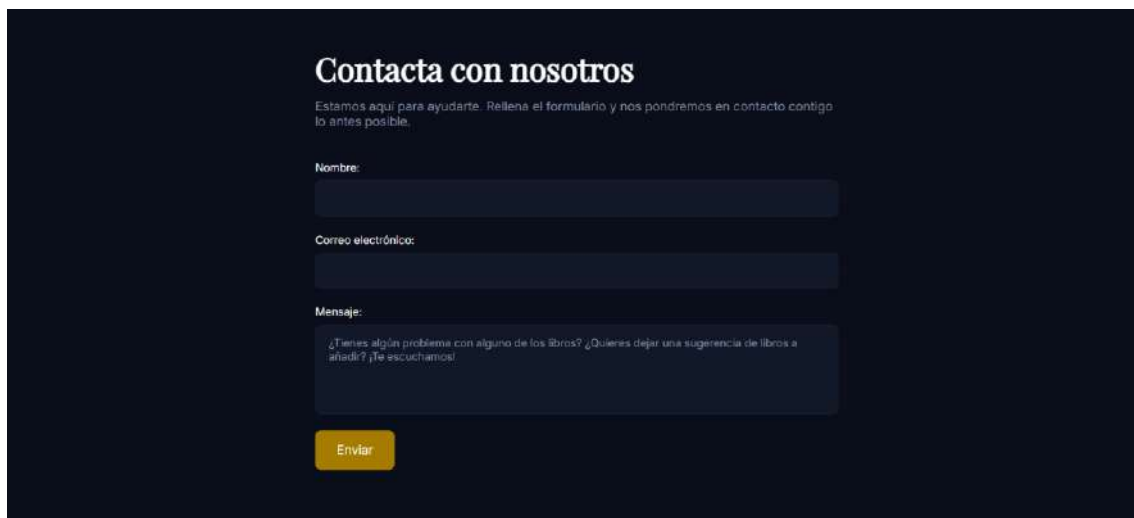
Página de inicio en escritorio



Página del catálogo en escritorio



Página de contacto en escritorio



Página de inicio en móvil



Página del catálogo en móvil



Página de contacto en móvil



The image shows a mobile application interface for 'Lectarium'. At the top, the brand name 'Lectarium' is displayed in a gold-colored serif font on the left, and a three-line hamburger menu icon is on the right. The main heading 'Contacta con nosotros' is centered in a large, white, serif font. Below this, a paragraph in a smaller white font reads: 'Estamos aquí para ayudarte. Rellena el formulario y nos pondremos en contacto contigo lo antes posible.' The form consists of three sections: 'Nombre:' followed by a dark blue rounded rectangular input field; 'Correo electrónico:' followed by a similar dark blue rounded rectangular input field; and 'Mensaje:' followed by a larger, dark blue rounded rectangular text area. Inside the text area, there is a line of light gray placeholder text: '¿Tienes algún problema con alguno de los libros? ¿Quieres dejar una sugerencia de libros a añadir? ¡Te escuchamos!'. At the very bottom of the form, a small portion of a yellow button is visible.

Lectarium

Contacta con nosotros

Estamos aquí para ayudarte. Rellena el formulario y nos pondremos en contacto contigo lo antes posible.

Nombre:

Correo electrónico:

Mensaje:

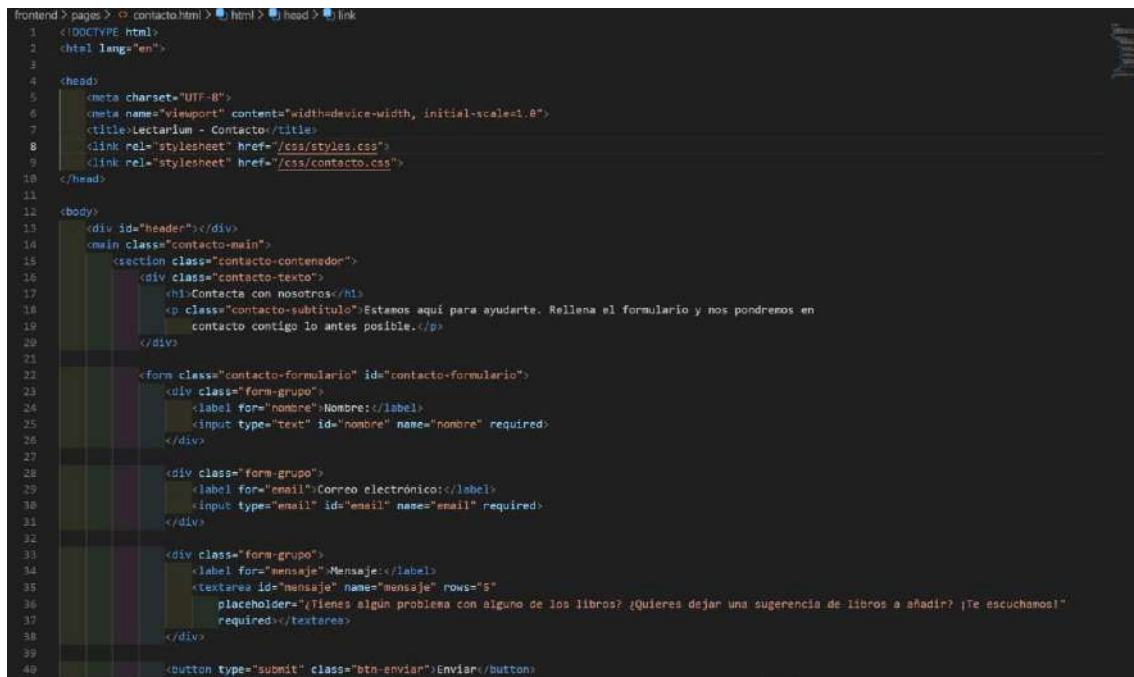
¿Tienes algún problema con alguno de los libros? ¿Quieres dejar una sugerencia de libros a añadir? ¡Te escuchamos!

TECNOLOGÍAS

Frontend**HTML5**

HTML5 es el lenguaje de marcado estándar para la creación de páginas web. Permite estructurar el contenido de manera semántica mediante etiquetas como <header>, <nav>, <main>, <section> y <footer>.

Uso en el proyecto: se ha utilizado para estructurar todas las páginas de Lectarium: index.html, catalogo.html, ficha-libro.html, login.html, registro.html y panel-usuario.html.



```

1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Lectarium - Contacto</title>
8   <link rel="stylesheet" href="/css/styles.css">
9   <link rel="stylesheet" href="/css/contacto.css">
10 </head>
11
12 <body>
13   <div id="header"></div>
14   <main class="contacto-main">
15     <section class="contacto-contenedor">
16       <div class="contacto-texto">
17         <h1>Contacta con nosotros</h1>
18         <p class="contacto-subtitulo">Estamos aquí para ayudarte. Rellena el formulario y nos pondremos en
19           contacto contigo lo antes posible.</p>
20       </div>
21
22       <form class="contacto-formulario" id="contacto-formulario">
23         <div class="form-grupo">
24           <label for="nombre">Nombre:</label>
25           <input type="text" id="nombre" name="nombre" required>
26         </div>
27
28         <div class="form-grupo">
29           <label for="email">Correo electrónico:</label>
30           <input type="email" id="email" name="email" required>
31         </div>
32
33         <div class="form-grupo">
34           <label for="mensaje">Mensaje:</label>
35           <textarea id="mensaje" name="mensaje" rows="5"
36             placeholder="¿Tienes algún problema con alguno de los libros? ¿Quieres dejar una sugerencia de libros a añadir? ¡Te escuchamos!"
37             required></textarea>
38         </div>
39
40         <button type="submit" class="btn-enviar">Enviar</button>

```

CSS3

CSS3 es el lenguaje de estilos que permite definir la apariencia visual de las páginas web. Incluye funcionalidades modernas como Flexbox, CSS Grid, animaciones y variables CSS.

Uso en el proyecto: se ha utilizado para implementar el diseño definido en Figma, incluyendo la paleta de colores, tipografía, layout responsive y efectos hover de las tarjetas de libros y botones.

```

frontend > css > # styles.css > @font-face
1  /* Estilos base comunes a todas las páginas */
2
3  @font-face {
4      font-family: "Playfair Display";
5      src: url(/assets/fonts/Playfair_Display/static/PlayfairDisplay-Regular.ttf);
6      font-weight: normal;
7      font-style: normal;
8  }
9
10 @font-face {
11     font-family: "Inter";
12     src: url(/assets/fonts/Inter/static/Inter_18pt-Regular.ttf);
13     font-weight: normal;
14     font-style: normal;
15 }
16
17 *{
18     margin: 0;
19     padding: 0;
20     box-sizing: border-box;
21 }
22
23 body {
24     min-height: 100vh;
25     display: flex;
26     flex-direction: column;
27     background-color: #0a0e1a;
28 }
29
30 main {
31     flex: 1;
32     margin: 30px 100px 30px 100px;
33 }
34
35 /* Header */
36
37 header {
38     padding: 10px 20px 10px 20px;
39     background-color: #0d1221;
40 }

```

JavaScript

JavaScript es el lenguaje de programación del navegador que permite añadir interactividad a las páginas web. Permite manipular el DOM, realizar peticiones a APIs y gestionar eventos de usuario.

Uso en el proyecto: se ha utilizado para conectar todas las páginas del frontend con la API REST del backend, cargando los libros dinámicamente en el catálogo y los carruseles, gestionando el proceso completo de autenticación desde el formulario hasta el almacenamiento del token, implementando la búsqueda y paginación en el catálogo, y gestionando las acciones del panel de usuario.

```

frontend > js > JS index.js > cargarDestacados
1 // Carga de libros
2
3 // Destacados
4 async function cargarDestacados() {
5   try {
6     const respuesta = await fetch(`${API_URL}/api/libros/destacados`);
7     const libros = await respuesta.json();
8
9     const carrusel = document.getElementById('carrusel-destacados');
10    carrusel.innerHTML = '';
11
12    libros.slice(0, 10).forEach(libro => {
13      carrusel.innerHTML += `
14        <div class="tarjeta-libro" onclick="window.location.href='/pages/ficha-libro.html?id=${libro.id_libro}'">
15          <div class="tarjeta-portada">
16            
18          </div>
19          <div class="tarjeta-info">
20            <h3 class="tarjeta-titulo">${libro.titulo}</h3>
21            <p class="tarjeta-autor">${libro.autor}</p>
22          </div>
23        </div>
24      `;
25    });
26  } catch (error) {
27    console.error('Error al cargar destacados:', error);
28  }
29 }
30
31
32 // Novedades
33 async function cargarNovedades() {
34   try {
35     const respuesta = await fetch(`${API_URL}/api/libros/novedades`);
36     const libros = await respuesta.json();
37
38     const carrusel = document.getElementById('carrusel-novedades');
39     carrusel.innerHTML = '';
40

```

Google Fonts

Google Fonts es un servicio gratuito de fuentes tipográficas web de Google.

Uso en el proyecto: se han utilizado las fuentes Playfair Display para títulos y encabezados, e Inter para el cuerpo del texto y la navegación.

Backend

Node.js

Node.js es un entorno de ejecución de JavaScript en el servidor, basado en el motor V8 de Google Chrome. Permite ejecutar JavaScript fuera del navegador y es especialmente eficiente para aplicaciones con muchas peticiones simultáneas gracias a su modelo asíncrono y no bloqueante. (OpenJS Foundation, s.f.)

La característica más destacada de Node.js es su modelo de entrada/salida no bloqueante y orientado a eventos. A diferencia de servidores tradicionales como Apache que crean un hilo por cada petición, Node.js gestiona todas las peticiones en un único hilo mediante un bucle de eventos, lo que lo hace extremadamente eficiente para aplicaciones con muchas peticiones concurrentes como las APIs REST.

PROMETEO

Uso en el proyecto: se ha utilizado la versión LTS de Node.js como entorno de ejecución del servidor de Lectarium. Toda la lógica del backend, desde la gestión de rutas hasta la comunicación con la base de datos, se ejecuta sobre Node.js.

Express.js

Express.js es un framework minimalista para Node.js que facilita la creación de servidores web y APIs REST. Proporciona un sistema de rutas, middlewares y gestión de peticiones HTTP. (OpenJS Foundation, s.f.)

Express simplifica considerablemente la creación de servidores web respecto a utilizar el módulo http nativo de Node.js. Proporciona un sistema de rutas para definir endpoints de forma declarativa, un sistema de middleware para procesar las peticiones antes de que lleguen al controlador, y una API sencilla para gestionar peticiones y respuestas HTTP.

Uso en el proyecto: se ha utilizado para definir todas las rutas de la API REST agrupadas por entidad en archivos de rutas separados, para gestionar el middleware de autenticación JWT aplicado a las rutas protegidas, para habilitar CORS y permitir las peticiones del frontend, y para parsear automáticamente el cuerpo de las peticiones en formato JSON con `express.json()`.

```
backend > src > JS index.js > ...
1  require('dotenv').config();
2
3  const express = require('express');
4  const conexion = require('./config/db');
5  const authRoutes = require('./routes/auth.routes');
6  const librosRoutes = require('./routes/libros.routes');
7  const prestamosRoutes = require('./routes/prestamos.routes');
8  const favoritosRoutes = require('./routes/favoritos.routes');
9
10 const app = express();
11
12 const cors = require('cors');
13 app.use(cors());
14
15 app.use(express.json());
16 app.use('/api/auth', authRoutes);
17 app.use('/api/libros', librosRoutes);
18 app.use('/api/prestamos', prestamosRoutes);
19 app.use('/api/favoritos', favoritosRoutes);
```

bcrypt

bcrypt es una librería de hashing de contraseñas basada en el algoritmo Blowfish. Es especialmente segura porque incorpora un factor de coste que hace que el proceso de hashing sea más lento, dificultando los ataques de fuerza bruta. (npm, s.f.)

Uso en el proyecto: se ha utilizado para hashear las contraseñas de los usuarios antes de almacenarlas en la base de datos y para verificarlas en el proceso de login.

JSON Web Token (JWT)

JWT es un estándar abierto definido en el RFC 7519 que establece un formato compacto y autocontenido para transmitir información de forma segura entre partes como un objeto JSON firmado digitalmente (Okta, s.f.). Un token JWT está compuesto por tres partes separadas por puntos: el header, que indica el algoritmo de firma; el payload, que contiene los datos del usuario; y la signature, que verifica la integridad del token.

La ventaja principal de JWT frente a las sesiones tradicionales basadas en cookies es que el servidor no necesita almacenar ningún estado. El token contiene toda la información necesaria para identificar al usuario y verificar su autenticidad, lo que hace que la API sea completamente stateless y escalable.

Uso en el proyecto: tras un login exitoso, el servidor genera un token JWT firmado con la clave secreta definida en el .env, que contiene el id_usuario y el rol del usuario con una expiración de 24 horas. El frontend almacena este token en localStorage y lo incluye en la cabecera authorization de cada petición a rutas protegidas. El middleware verificarToken comprueba la validez del token en cada petición antes de permitir el acceso al controlador.

dotenv

dotenv es una librería de Node.js que permite cargar variables de entorno desde un archivo .env al objeto process.env en tiempo de ejecución (npm, s.f.). Esta práctica es un estándar en el desarrollo de aplicaciones web para separar la configuración del código, siguiendo los principios de la metodología Twelve-Factor App.

Uso en el proyecto: se ha utilizado para almacenar de forma segura y separada del código las credenciales de conexión a la base de datos (host, puerto, usuario, contraseña y nombre de la base de datos), la clave secreta para firmar los tokens JWT y el puerto del servidor.

PROMETEO

El archivo .env está incluido en el .gitignore para evitar que datos sensibles se suban al repositorio de GitHub.

nodemon

nodemon es una herramienta de desarrollo para aplicaciones Node.js que monitoriza los cambios en los archivos del proyecto y reinicia automáticamente el servidor cuando detecta modificaciones (Nodemon, s.f.). Sin nodemon, sería necesario detener y reiniciar manualmente el servidor con cada cambio en el código.

Uso en el proyecto: se ha configurado como script de desarrollo en el package.json del proyecto mediante el comando npm run dev. Solo se utiliza durante el desarrollo local y no se incluye en el entorno de producción.

Base de datos

MySQL

MySQL es un sistema de gestión de bases de datos relacional de código abierto, ampliamente utilizado en aplicaciones web. Utiliza el lenguaje SQL para la definición y manipulación de datos. (Oracle, s.f.)

Uso en el proyecto: se ha utilizado para almacenar y gestionar toda la información de Lectarium: usuarios, libros, géneros, préstamos y favoritos.

MySQL Workbench

MySQL Workbench es una herramienta visual oficial de MySQL que permite diseñar, modelar y administrar bases de datos. (Oracle, s.f.)

Uso en el proyecto: se ha utilizado para diseñar el diagrama entidad-relación, crear la base de datos y ejecutar consultas durante el desarrollo.

mysql2

mysql2 es el conector oficial de MySQL para Node.js, que permite ejecutar consultas SQL desde el backend. (npm, s.f.)

Uso en el proyecto: se ha utilizado para conectar el servidor Node.js con la base de datos MySQL y ejecutar todas las consultas de la API.

Diseño y prototipado

Figma

Figma es una herramienta de diseño de interfaces web y móviles colaborativa basada en la nube. Permite crear wireframes, prototipos y sistemas de diseño. (Figma, s.f.)

Uso en el proyecto: se ha utilizado para diseñar el prototipo completo de Lectarium antes de comenzar el desarrollo del frontend, definiendo la paleta de colores, tipografía y layout de todas las pantallas.

Draw.io

Draw.io es una herramienta gratuita de creación de diagramas online. (diagrams.net, s.f.)

Uso en el proyecto: se ha utilizado para crear el diagrama entidad-relación y el diagrama relacional extendido de la base de datos.

Entorno de desarrollo y control de versiones

Visual Studio Code

Visual Studio Code es un editor de código fuente gratuito y de código abierto desarrollado por Microsoft. Es altamente extensible mediante plugins. (Microsoft, s.f.)

Uso en el proyecto: se ha utilizado como editor principal para el desarrollo de todo el código del proyecto.

Git y GitHub

Git es un sistema de control de versiones distribuido y GitHub es la plataforma web que aloja repositorios Git. (GitHub, s.f.)

Uso en el proyecto: se han utilizado para el control de versiones del proyecto, permitiendo trabajar en equipo de forma coordinada y mantener un historial de cambios.

Thunder Client

Thunder Client es una extensión de Visual Studio Code que permite realizar peticiones HTTP para probar APIs REST directamente desde el editor. (Thunder Client, s.f.)

PROMETEO

Uso en el proyecto: se ha utilizado durante el desarrollo del backend para probar todos los endpoints de la API antes de conectarlos con el frontend.

METODOLOGÍA

Metodología de desarrollo

Para el desarrollo del proyecto Lectarium hemos seguido una metodología ágil basada en iteraciones cortas con objetivos bien definidos para cada sesión de trabajo. Esta metodología, inspirada en los principios del desarrollo ágil de software, nos ha permitido adaptarnos a los cambios y problemas surgidos durante el desarrollo sin perder el hilo del progreso general.

El desarrollo se ha dividido en cuatro fases secuenciales, cada una de las cuales debía estar completada antes de pasar a la siguiente, siguiendo el principio de construcción progresiva:

- Fase 1 — Planificación y diseño de la base de datos: definición del modelo de datos, creación de los diagramas ER y relacional, implementación del esquema SQL e inserción de datos de prueba.
- Fase 2 — Desarrollo del backend: configuración del servidor Node.js con Express, implementación de la API REST con todos los endpoints necesarios, sistema de autenticación con bcrypt y JWT, y middleware de protección de rutas.
- Fase 3 — Diseño de la interfaz en Figma: prototipado visual completo de todas las pantallas, definición de la paleta de colores, tipografía y componentes reutilizables.
- Fase 4 — Desarrollo del frontend: maquetación HTML y CSS de todas las páginas, implementación de la funcionalidad JavaScript y conexión con la API REST del backend.

El control de versiones se ha gestionado mediante Git y GitHub. Cada desarrolladora ha trabajado en su propia rama de desarrollo y se han realizado merges a la rama main al finalizar cada funcionalidad, manteniendo un historial de commits claro y descriptivo que permite rastrear el progreso real del proyecto.

PROMETEO

Planificación inicial estimada

Fase	Tareas principales	Horas estimadas
Planificación y BD	Diagramas ER, script SQL, datos de prueba	8h
Backend	Configuración servidor, API REST, autenticación, middleware	16h
Diseño Figma	Prototipo de 6 pantallas, paleta, tipografía	8h
Frontend HTML/CSS	Maquetación de todas las páginas	12h
Frontend JavaScript	Conexión API, funcionalidad, paginación	12h
Responsive	Media queries para móvil y tablet	6h
Memoria	Redacción completa de la memoria del proyecto	16h
TOTAL		78h

Planificación real

Fase	Tareas principales	Horas estimadas
Planificación y BD	Diagramas ER, script SQL, datos de prueba	6h
Backend	Configuración servidor, API REST, autenticación, middleware	20h
Diseño Figma	Prototipo de 6 pantallas, paleta, tipografía	5h
Frontend HTML/CSS	Maquetación de todas las páginas	24h
Frontend JavaScript	Conexión API, funcionalidad, paginación	14h
Responsive	Media queries para móvil y tablet	6h
Memoria	Redacción completa de la memoria del proyecto	18h
TOTAL		93h

Presupuesto

El coste del desarrollo de Lectarium se ha calculado en base a las horas de trabajo dedicadas por cada miembro del equipo, aplicando una tarifa de 15 euros por hora correspondiente al perfil de desarrolladora web junior en prácticas. Este presupuesto es orientativo y refleja el valor económico del trabajo realizado.

Concepto	Desarrolladora	Horas	Tarifa/h	Total
Base de datos y diagramas	Andrea / Cristina	6h	15€	90€

PROMETEO

Desarrollo backend	Andrea / Cristina	20h	15€	300€
Diseño Figma	Andrea / Cristina	5h	15€	75€
Desarrollo frontend	Andrea / Cristina	24h	15€	360€
Responsive	Andrea / Cristina	6h	15€	90€
Memoria y documentación	Andrea / Cristina	18h	15€	270€
TOTAL		93h		1.395€

Las herramientas utilizadas en el proyecto son todas gratuitas o de código abierto, por lo que el coste de licencias de software es 0 euros. El coste total del proyecto corresponde íntegramente al trabajo de desarrollo.

CONCLUSIONES

El desarrollo de Lectarium ha supuesto una experiencia de aprendizaje completa y enriquecedora que ha puesto a prueba y consolidado los conocimientos adquiridos a lo largo del ciclo formativo de Desarrollo de Aplicaciones Web. A través de este proyecto hemos trabajado con las tecnologías más demandadas del sector web actual, desarrollando una aplicación real, funcional y con valor de uso efectivo.

Uno de los aprendizajes más relevantes ha sido la importancia de planificar cuidadosamente la base de datos antes de escribir cualquier línea de código. Las decisiones tomadas en el diseño del esquema SQL, como la estructura de las tablas, las relaciones entre ellas y las restricciones de integridad, han condicionado todo el desarrollo posterior. Cambiar el esquema a mitad del desarrollo implica modificar los controladores del backend, las consultas SQL y a veces incluso el frontend, por lo que una planificación sólida en esta fase ahorra un tiempo considerable.

El desarrollo del backend con Node.js y Express.js nos ha proporcionado una comprensión profunda de cómo funciona una API REST: el ciclo de vida de una petición HTTP, el papel del middleware, la autenticación basada en tokens JWT, la seguridad en el almacenamiento de contraseñas con bcrypt y la importancia de las consultas parametrizadas para prevenir inyecciones SQL.

El trabajo en equipo ha sido fundamental para completar el proyecto en los plazos establecidos. El uso de Git y GitHub para el control de versiones nos ha permitido trabajar en paralelo sobre diferentes partes del proyecto sin conflictos, y mantener un historial claro del progreso que puede contrastarse con la planificación inicial.

La fase de diseño en Figma, aunque inicialmente puede parecer un paso adicional antes de empezar a programar, ha demostrado ser enormemente valiosa. Tener un prototipo visual completo como referencia durante el desarrollo del frontend ha acelerado el proceso de maquetación y ha reducido la necesidad de tomar decisiones de diseño sobre la marcha, permitiendo centrarnos en la implementación técnica.

Como conclusión general, consideramos que Lectarium es un proyecto que demuestra de forma sólida nuestra capacidad para desarrollar aplicaciones web completas desde cero,

PROMETEO

aplicando buenas prácticas de desarrollo como la arquitectura en capas, la separación de responsabilidades, la seguridad en la autenticación y el diseño responsive. Estamos satisfechas con el resultado obtenido y confiadas en que los conocimientos y habilidades desarrollados durante este proyecto serán directamente aplicables en nuestra futura vida profesional como desarrolladoras web.

TRABAJOS FUTUROS

Una vez completado el proyecto en su versión actual, se identifican las siguientes líneas de mejora y ampliación que podrían desarrollarse en futuras versiones de Lectarium:

Sistema de valoraciones

Implementar la posibilidad de que los usuarios puntúen los libros leídos mediante un sistema de 1 a 5 estrellas con comentario opcional. La valoración media de cada libro se mostraría en la tarjeta del catálogo y en la ficha del libro. La base de datos ya cuenta con la tabla valoraciones preparada para esta funcionalidad.

Calendario de préstamos

Añadir una vista de calendario en el panel de usuario donde el usuario pueda visualizar de forma gráfica las fechas de vencimiento de todos sus préstamos activos. Esta funcionalidad facilitaría la planificación de la lectura y reduciría los préstamos vencidos por olvido.

Panel de administración

Desarrollar una interfaz de administración accesible exclusivamente para usuarios con rol de administrador que permita gestionar el catálogo completo de libros, añadiendo, editando o eliminando títulos, y administrar los usuarios y préstamos sin necesidad de acceder directamente a la base de datos.

Notificaciones por email

Implementar un sistema de notificaciones automáticas por correo electrónico utilizando la librería Nodemailer de Node.js. Los usuarios recibirían avisos cuando un préstamo esté próximo a vencer (por ejemplo, tres días antes) y una confirmación por email al registrarse en la plataforma.

Búsqueda avanzada y ordenación

Ampliar las capacidades de búsqueda del catálogo añadiendo filtros por año de publicación, y opciones de ordenación por título, autor, valoración media o fecha de adición al catálogo. Esto mejoraría significativamente la experiencia de usuario para catálogos grandes.

Lector de libros integrado

Integrar un lector de PDFs directamente en la plataforma, eliminando la necesidad de descargar el archivo para leerlo. El lector mostraría el PDF con controles de navegación entre páginas, zoom y modo pantalla completa, todo dentro de la misma interfaz de Lectarium.

REFERENCIAS

diagrams.net. (s.f.). Draw.io Documentation. Recuperado de <https://www.diagrams.net>

Figma. (s.f.). Figma Documentation. Recuperado de <https://help.figma.com>

GitHub. (s.f.). GitHub Documentation. Recuperado de <https://docs.github.com>

Hart, M. (s.f.). Project Gutenberg. Recuperado de <https://www.gutenberg.org>

Microsoft. (s.f.). Visual Studio Code Documentation. Recuperado de <https://code.visualstudio.com/docs>

Ministerio de Cultura y Deporte. (s.f.). eBiblio. Recuperado de <https://ebiblio.es>

Nodemon. (s.f.). Nodemon Documentation. Recuperado de <https://nodemon.io>

npm. (s.f.). bcrypt. Recuperado de <https://www.npmjs.com/package/bcrypt>

npm. (s.f.). dotenv. Recuperado de <https://www.npmjs.com/package/dotenv>

npm. (s.f.). mysql2. Recuperado de <https://www.npmjs.com/package/mysql2>

Okta. (s.f.). Introduction to JSON Web Tokens. Recuperado de <https://jwt.io/introduction>

OpenJS Foundation. (s.f.). Express.js Documentation. Recuperado de <https://expressjs.com>

OpenJS Foundation. (s.f.). Node.js Documentation. Recuperado de <https://nodejs.org/en/docs>

Oracle Corporation. (s.f.). MySQL Documentation. Recuperado de <https://dev.mysql.com/doc>